



ONTAP Automation: Getting Started with Ansible

March 2026 | LD01301 Version 1.2.1

TABLE OF CONTENTS

1 Introduction.....3

2 Lab Activities..... 4

2.1 Getting Familiar with Ansible and NetApp ONTAP Modules.....4

 2.1.1 Ansible Configuration and Inventory.....4

 2.1.2 Ansible Commands and Modules..... 10

 2.1.3 Write an Ansible Playbook Using an ONTAP Module.....15

2.2 Write a Playbook to Provision ONTAP Storage.....21

 2.2.1 Use Playbook Variables to Simplify Tasks..... 22

 2.2.2 Add NFS Provisioning Tasks.....31

 2.2.3 Use Conditionals to Incorporate CIFS Into Your Playbook..... 39

2.3 Make Playbooks More Useful with Ansible Roles.....46

3 Lab Summary..... 52

4 Additional Information..... 53

 4.1 Lab Environment..... 53

 4.2 References.....53

 4.3 Version History.....54

Legal Notices..... 55

1 Introduction

The typical IT department is facing growing complexity in their environment while at the same time attempting to improve efficiency. This is why most organization have found it increasingly necessary to embrace automation. The goal is to speed up common tasks, free up time for the IT staff, and to prevent mistakes that are commonplace, especially during lengthy manual procedures. Automation provides the ability to scale existing processes and allows admins to be more confident when making system changes because those changes become more predictable and refineable over time.

Ansible Automation is a framework for automating across IT operations, and for many organizations has become the standard enterprise automation method. Ansible has some qualities that make it uniquely convenient and useful to administrators:

- **Agentless:** Ansible is agentless, meaning nothing needs to be set up on the managed host. Instead, Ansible will connect to a specified host and execute its tasks on the host itself.
- **Human Friendly Syntax:** Ansible uses playbooks to structure its automation tasks, which are executed procedurally. These playbooks are written in an easy-to-read language called YAML, and act as descriptions of the desired state of some part of an IT application infrastructure.
- **Extensible Ecosystem:** Ansible's capabilities come from pieces of code called modules and plugins. This code can control resources and execute commands. Because anyone can write these modules, Ansible works across a wide variety of IT components including servers, containers, firewalls, load balancers, network devices, storage, and even cloud components.

These and other qualities make Ansible an ideal solution for automating across different teams and environments for things like orchestration, configuration management, continuous delivery, security and compliance.

With a significant portion of IT administrative workflows involving storage management tasks, NetApp ONTAP modules for Ansible are a powerful tool in achieving these goals. In this lab, you leverage these modules to automate storage provisioning with considerations for things like protocol and access control.

You will learn the basics of Ansible by creating a simple playbook that uses NetApp ONTAP modules for Ansible. As you go through the lab, you will evolve your simple playbook into a more capable and flexible provisioning workflow.

Some of the benefits of managing ONTAP with Ansible include:

- Commonly repeated tasks are sped up, simplified, and less error-prone allowing an IT department to accomplish more with fewer people.
- Ansible playbooks and roles make it easier to manage changes across multiple teams and environments.
- Ansible can integrate an automated storage workflow with third-party applications, such as an IT ticketing system, which allows for end-user initiated automation tasks that require no hands-on administration at all.
- With reliable automation, large-scale changes and deployments can take a fraction of the time and have much more predictable results.

2 Lab Activities

This lab explores the following topics:

- [Getting Familiar with Ansible and NetApp ONTAP Modules](#)
 - [Ansible Configuration and Inventory](#)
 - [Ansible Commands and Modules](#)
 - [Write an Ansible Playbook Using an ONTAP Module](#)
- [Write a Playbook to Provision ONTAP Storage](#)
 - [Use Playbook Variables to Simplify Tasks](#)
 - [Add NFS Provisioning Tasks](#)
 - [Use Conditionals to Incorporate CIFS Into Your Playbook](#)
- [Make Playbooks More Useful with Ansible Roles](#)

2.1 Getting Familiar with Ansible and NetApp ONTAP Modules

Before working with Ansible to provision storage in ONTAP, you need to understand what Ansible is and how it operates within your environment.

In this section you explore the follow activities:

Table 1:

Activities
Ansible Configuration and Inventory
Add NFS Provisioning Tasks
Write an Ansible Playbook Using an ONTAP Module

2.1.1 Ansible Configuration and Inventory

While there are many configuration options when setting up Ansible, getting started with a basic configuration is simple. The host from which you run Ansible commands is referred to as the control node. In your lab, Ansible has already been installed to the control node which is an Ubuntu Linux server.



Tip: Ansible can be easily installed with a package manager such as pip. See the [References section](#) for a guide on how to install Ansible.

In this lab activity, you use VSCodium to connect to the Ansible control node and examine Ansible's configuration on that host.

1. Double-click the **VSCodium** shortcut on the Jumphost desktop to open VSCodium.

2. VSCodium should open to an existing workspace called “Ansible Workspace”. VSCodium utilizes an SSH connection to the ansible host, so the explorer panel shows the home directory for the ansible user on the ansible host.

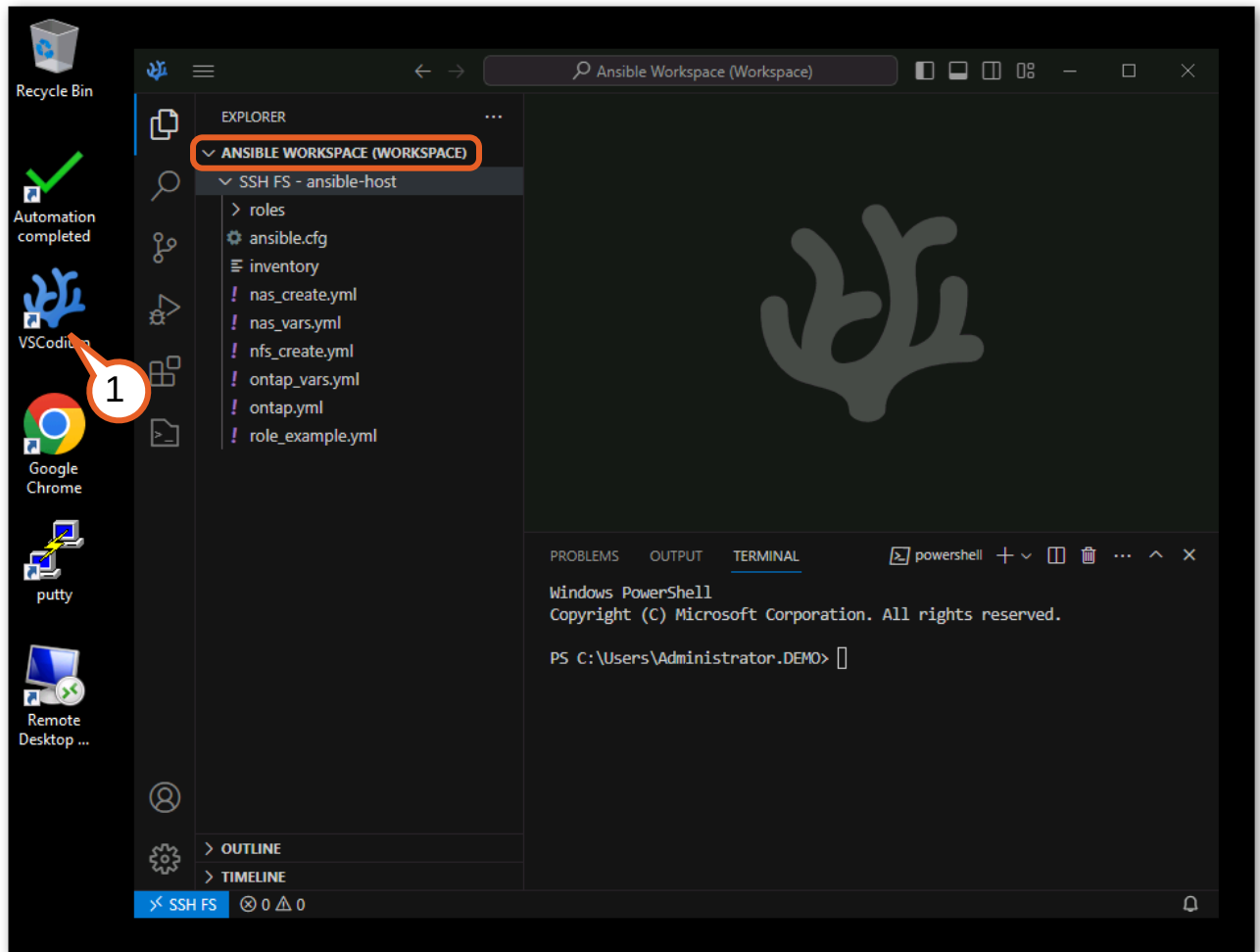


Figure 2-1:

3. Right-click “ansible-host” and select **Open remote SSH terminal** from the menu to open an SSH terminal to the ansible host.

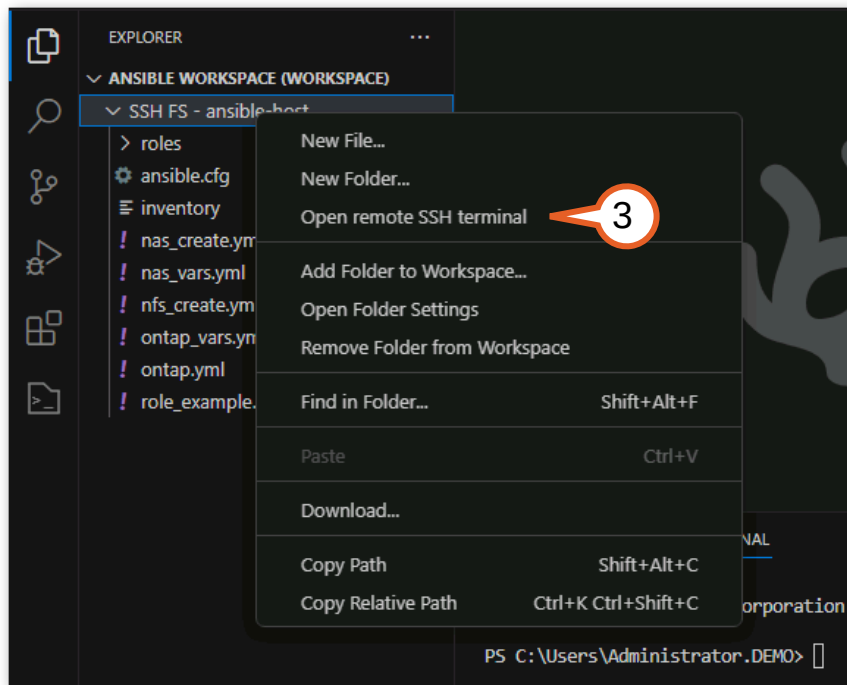
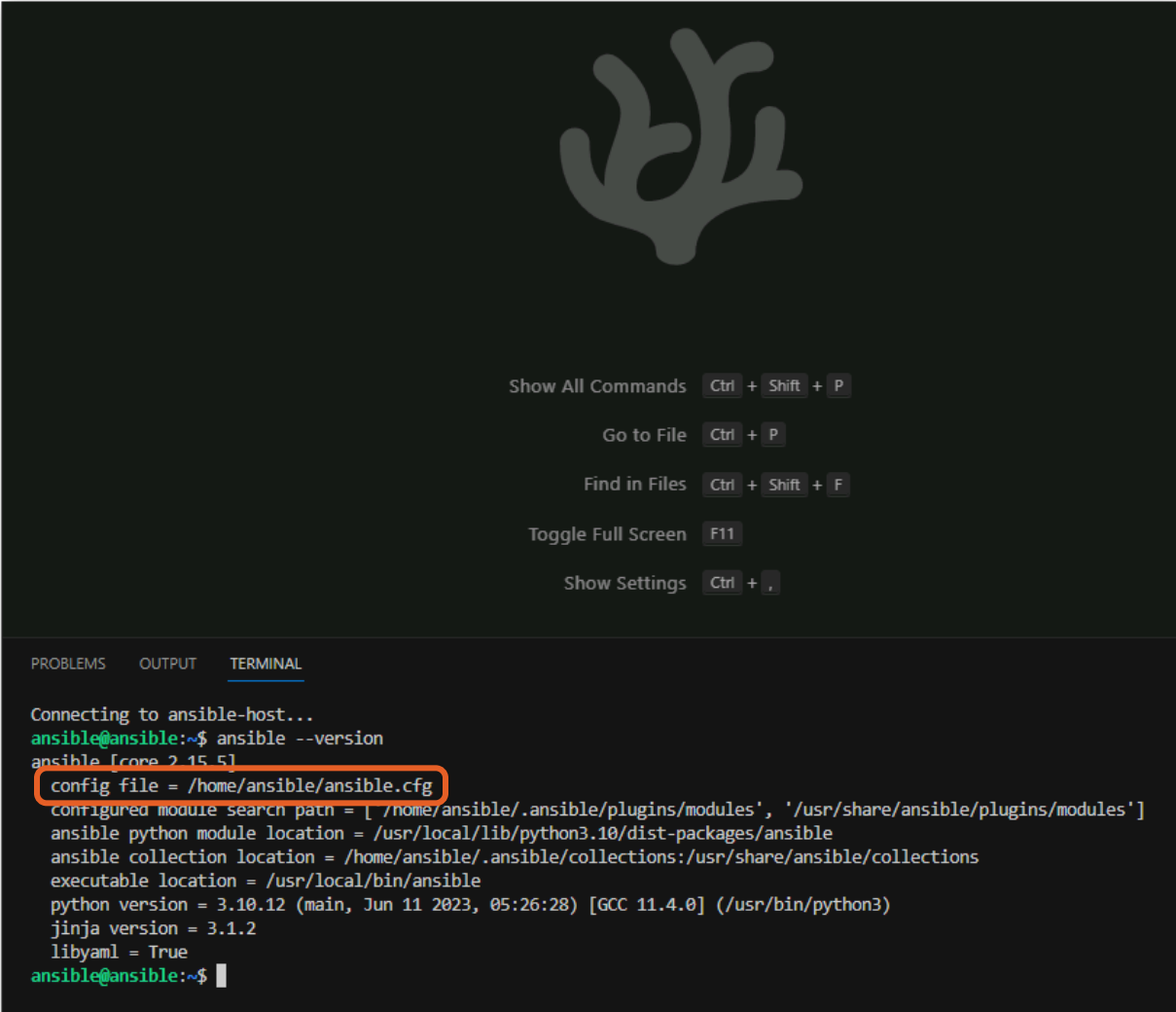


Figure 2-2:

4. The terminal opens up in a small panel at the bottom.
Enter the following command into the terminal window:

```
ansible --version
```

5. Notice that information about the Ansible installation is displayed, including a reference to a config file.



The screenshot shows a terminal window with a dark background. At the top, there is a large, faint, stylized logo. Below the logo, there are several keyboard shortcuts listed: "Show All Commands" (Ctrl + Shift + P), "Go to File" (Ctrl + P), "Find in Files" (Ctrl + Shift + F), "Toggle Full Screen" (F11), and "Show Settings" (Ctrl + ,). Below these shortcuts, there are tabs labeled "PROBLEMS", "OUTPUT", and "TERMINAL". The "TERMINAL" tab is active, showing the following text:

```
Connecting to ansible-host...
ansible@ansible:~$ ansible --version
ansible [core 2.15.5]
  config file = /home/ansible/ansible.cfg
  configured module search path = [ /home/ansible/.ansible/plugins/modules, '/usr/share/ansible/plugins/modules' ]
  ansible python module location = /usr/local/lib/python3.10/dist-packages/ansible
  ansible collection location = /home/ansible/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/local/bin/ansible
  python version = 3.10.12 (main, Jun 11 2023, 05:26:28) [GCC 11.4.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
ansible@ansible:~$
```

The line "config file = /home/ansible/ansible.cfg" is highlighted with a red rectangle.

Figure 2-3:

6. Click **ansible.cfg** from the explorer panel on the left.

7. The `ansible.cfg` file opens in a tab on the right.

Once installed, the behavior of Ansible can be customized by modifying settings in Ansible's INI-style configuration file. Ansible will select its configuration file from one of the following possible locations on the control node, in order of precedence:

- `ANSIBLE_CONFIG` (environment variable if set)
- `ansible.cfg` (in the current directory)
- `~/.ansible.cfg` (in the home directory)
- `/etc/ansible/ansible.cfg` (installed as Ansible default)

In your lab environment, this `ansible.cfg` file has already been created and filled with the necessary details in the home directory of your user on the control node.

There are multiple configuration flags provided. Most of them are not of interest here, but make sure to note the last line where the location of an inventory file is provided.

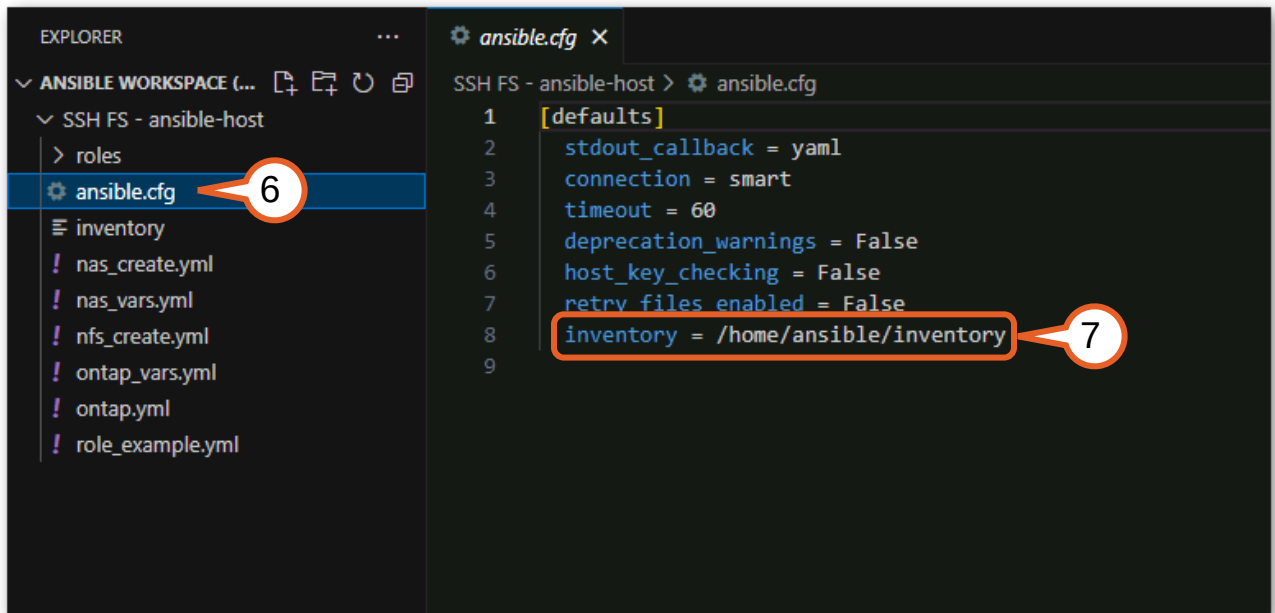


Figure 2-4:

8. Click **inventory** from the explorer panel to open it.
9. Ansible is designed to allow you to perform automation tasks on multiple hosts at once, and in order to do this you need to provide an inventory file which defines a list of hosts to be managed (also called managed nodes) from the Ansible control node.

You can organize your inventory of hosts into groups. For instance, in this inventory file there is a group for Linux hosts called "linux". You can perform automation tasks against groups of hosts like linux.

10. You can also have variables assigned to a host or a group of hosts that can be referenced in your automation tasks.

In this inventory there are variables set for all hosts in the section labeled "[all:vars]". This includes the `ansible_user` variable, which is the user ID that ansible commands and playbooks will be running as. You will

explore variables in greater detail later on in this lab, but for now know that by setting these variables here, Ansible can connect to these hosts without explicitly asking for credential information.

Note: The inventory can contain more information. For example, if you have hosts that run on non-standard SSH ports you can put the port number after the hostname with a colon. Or you could define host names within Ansible that map to the real IP or hostname.

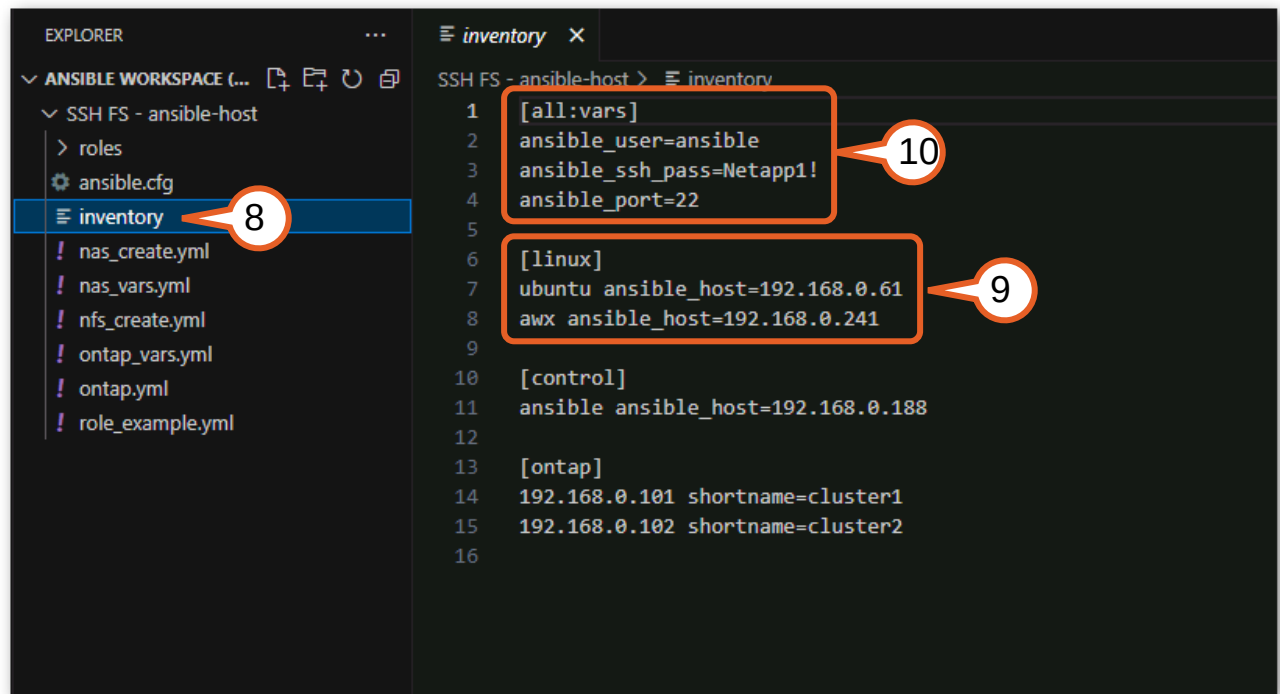


Figure 2-5:

11. To reference inventory hosts, you supply a host pattern to the ansible command. Ansible has a `--list-hosts` option which can be useful for clarifying which managed hosts or group of hosts are referenced by the host pattern in an ansible command.

In VSCode's terminal pane, enter the following command:

```
ansible linux --list-hosts
```

12. Notice that the result of the command is a list of hosts in the Linux group.

Tip: You can put systems in more than one group. For instance, the ansible host is in both the control group and the Linux group.

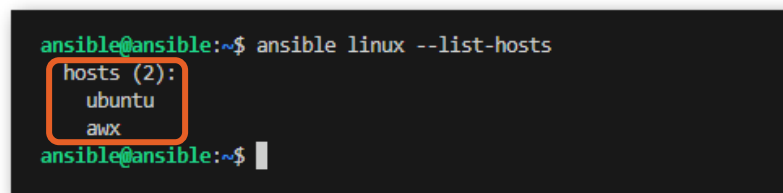


Figure 2-6:

Now that you understand a bit about Ansible configuration and how Ansible is configured on the ansible host, you are ready to start using Ansible in the next activity.

2.1.2 Ansible Commands and Modules

At a basic level, to use Ansible to perform tasks you need to specify one or more hosts and modules, along with any options. The modules perform the action you want to perform, and the hosts are where the tasks are performed. While usually these elements are specified within playbooks (which you will explore in the next activity), you can execute Ansible tasks using ad-hoc commands. These are single Ansible tasks that you run from the command-line of the control host.

To get an idea of how this works, you will execute some ad-hoc commands using some common Ansible modules, starting with the ping module. You can think of a module as a tool that is designed to accomplish a specific task, and the ping module's task is to ensure target hosts are responsive. The code that makes up the ping module does this by connecting to the managed host, executing a small script there, and then collecting the results. This procedure ensures that the managed host is reachable and that Ansible is able to execute commands properly on it.

1. To use the ping module in your ad-hoc command, you must specify the module using the “-m” argument which is the short form of “--module-name”.



Tip: Like many Linux commands, ansible allows for long-form options as well as short-form. This lab uses the short-form options throughout.

In the terminal window to the ansible host, execute this ad-hoc command to ping hosts in the Linux group of your inventory.

```
ansible linux -m ping
```



Important: If at any time your terminal window becomes unresponsive, you can open a new connection by right-clicking ansible-host from the explorer panel and clicking “Open remote SSH terminal”.

2. You can see each node from your inventory'd node group reports the successful execution, displaying a result of “SUCCESS” and a value of “pong”.



Tip: Sometimes, especially when running a command against many hosts, it is useful to limit the output to a single line per host. Enter the command again with the “-o” option (short for “--one-line”).

```
ansible@ansible:~$ ansible linux -m ping
ubuntu | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
awx | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
ansible@ansible:~$
```

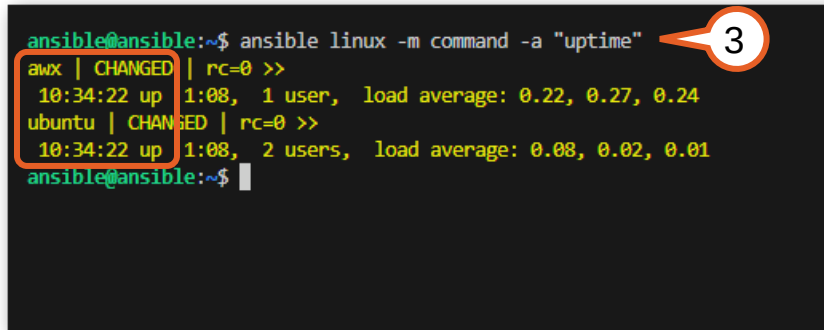
Figure 2-7:

3. There is an Ansible module called “command” that allows you to execute arbitrary Linux commands on a managed host. When using this module, you need to pass an argument to the module specifying which Linux command to execute on the managed host.

To do this with an ad-hoc command, add “-a” followed by the Linux command. Try this command in your terminal window:

```
ansible linux -m command -a "uptime"
```

4. The result is the uptime for all of the managed hosts in the Linux group.



```
ansible@ansible:~$ ansible linux -m command -a "uptime"
awx | CHANGED | rc=0 >>
10:34:22 up 1:08, 1 user, load average: 0.22, 0.27, 0.24
ubuntu | CHANGED | rc=0 >>
10:34:22 up 1:08, 2 users, load average: 0.08, 0.02, 0.01
ansible@ansible:~$
```

Figure 2-8:

5. Finally, use the “file” module to create a directory on the host “ubuntu”. In this case the arguments string will include module parameters defined with a syntax that looks like “key=value”.



Tip: Module arguments can also use JSON formatting.

Run the following, but expect an error:

```
ansible ubuntu -m file -a "path=/var/new_dir state=directory"
```

6. As expected, this produces an error. The desired result was to create a new directory, called “new_dir” within the path “/var” on the ubuntu host. Because var is owned by the root user, the ansible user does not have permission to create a directory within “/var”. That is why the command failed, indicated with a “permission denied” message.



```
ansible@ansible:~$ ansible ubuntu -m file -a "path=/var/new_dir state=directory"
ubuntu | FAILED! => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "msg": "There was an issue creating /var/new_dir as requested: [Errno 13] Permission denied: b'/var/new_dir'",
  "path": "/var/new_dir"
}
ansible@ansible:~$
```

Figure 2-9:

7. Ansible has a feature called “become” which instructs Ansible to use sudo to run the command on the managed host. You enable this with ad-hoc commands by using the parameter “-b”.

Add the parameter to the command and run it again.

```
ansible ubuntu -m file -a "path=/var/new_dir state=directory" -b
```

8. This time the command is a success.

```
ansible@ansible:~$ ansible ubuntu -m file -a "path=/var/new_dir state=directory" -b
ubuntu | CHANGED -> {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": true,
  "gid": 0,
  "group": "root",
  "mode": "0755",
  "owner": "root",
  "path": "/var/new_dir",
  "size": 4096,
  "state": "directory",
  "uid": 0
}
ansible@ansible:~$
```

Figure 2-10:

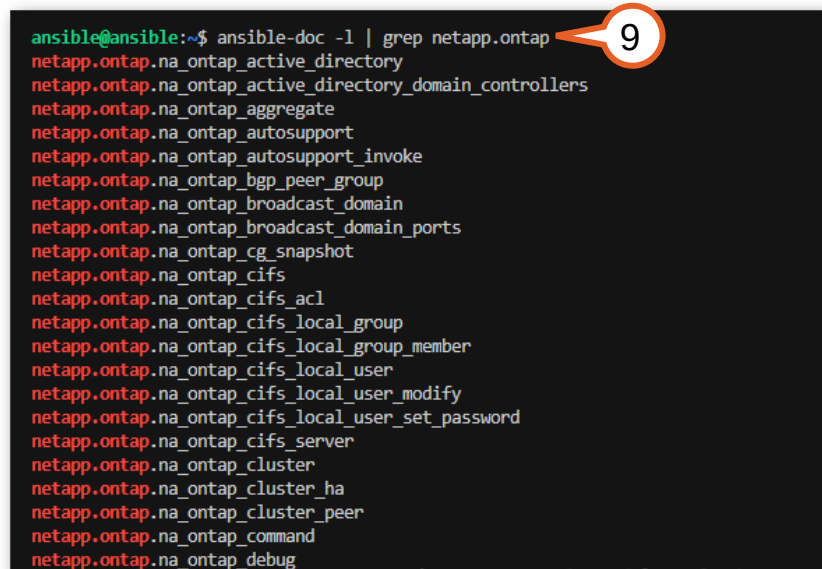


Tip: Ansible will connect to the machines using your current user name (ansible in this case), just like SSH would. To override the remote user name, you can use the `-u` parameter.

9. In this activity you have used three modules, but Ansible comes with many more modules by default. You can see a list of available modules using the command “ansible-doc -l”, however that list is very long.

Many NetApp modules have been added to your lab environment in advance by installing the netapp.ontap collection. Use the “ansible-doc” command and filter on “netapp.ontap” using Linux’s grep command to see a list of netapp.ontap modules:

```
ansible-doc -l | grep netapp.ontap
```



```
ansible@ansible:~$ ansible-doc -l | grep netapp.ontap
netapp.ontap.na_ontap_active_directory
netapp.ontap.na_ontap_active_directory_domain_controllers
netapp.ontap.na_ontap_aggregate
netapp.ontap.na_ontap_autosupport
netapp.ontap.na_ontap_autosupport_invoke
netapp.ontap.na_ontap_bgp_peer_group
netapp.ontap.na_ontap_broadcast_domain
netapp.ontap.na_ontap_broadcast_domain_ports
netapp.ontap.na_ontap_cg_snapshot
netapp.ontap.na_ontap_cifs
netapp.ontap.na_ontap_cifs_acl
netapp.ontap.na_ontap_cifs_local_group
netapp.ontap.na_ontap_cifs_local_group_member
netapp.ontap.na_ontap_cifs_local_user
netapp.ontap.na_ontap_cifs_local_user_modify
netapp.ontap.na_ontap_cifs_local_user_set_password
netapp.ontap.na_ontap_cifs_server
netapp.ontap.na_ontap_cluster
netapp.ontap.na_ontap_cluster_ha
netapp.ontap.na_ontap_cluster_peer
netapp.ontap.na_ontap_command
netapp.ontap.na_ontap_debug
```

Figure 2-11:

Notice there are many modules in the collection netapp.ontap. Each of these modules can perform a specific set of tasks within your Ansible playbooks, giving you flexibility in how you approach storage management tasks.



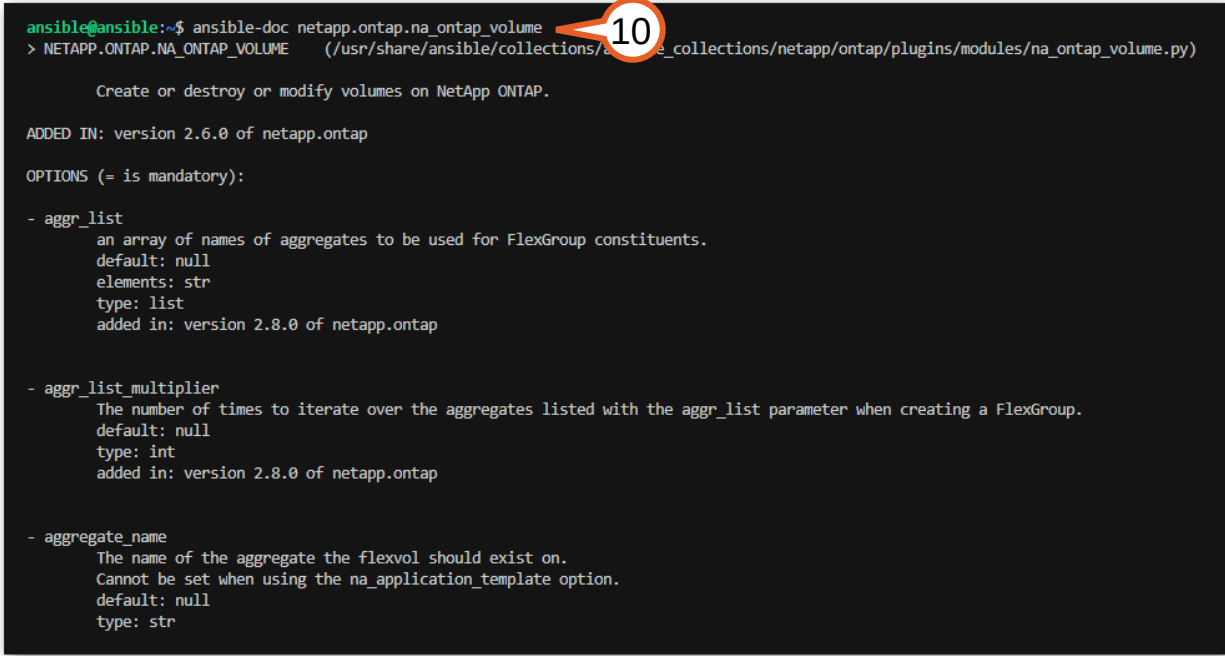
Note: Ansible collections are packages of Ansible content, containing modules, plugins, and roles (which will be discussed later in this lab). Collections make it easy to share and reuse Ansible content and can be installed multiple ways. A common method is retrieving collections from Ansible Galaxy, which is the public repository of community contributed collections.

You can add the NetApp ONTAP module collection to your environment by using the ansible-galaxy command like this: `ansible-galaxy collection install netapp.ontap`.

10. You can find documentation for a specific module using `ansible-doc` by specifying the module in the command:

```
ansible-doc netapp.ontap.na_ontap_volume
```

The displayed documentation includes what the module does as well as the module's parameters and usage examples. In `ansible-doc`, optional parameters are denoted with a "-" while mandatory parameters are marked by a "=". At the end of the documentation there are example tasks using the module to target various configurations.



```
ansible@ansible:~$ ansible-doc netapp.ontap.na_ontap_volume
> NETAPP.ONTAP.NA_ONTAP_VOLUME (/usr/share/ansible/collections/ansible_collections/netapp/ontap/plugins/modules/na_ontap_volume.py)

    Create or destroy or modify volumes on NetApp ONTAP.

ADDED IN: version 2.6.0 of netapp.ontap

OPTIONS (= is mandatory):

- aggr_list
  an array of names of aggregates to be used for FlexGroup constituents.
  default: null
  elements: str
  type: list
  added in: version 2.8.0 of netapp.ontap

- aggr_list_multiplier
  The number of times to iterate over the aggregates listed with the aggr_list parameter when creating a FlexGroup.
  default: null
  type: int
  added in: version 2.8.0 of netapp.ontap

- aggregate_name
  The name of the aggregate the flexvol should exist on.
  Cannot be set when using the na_application_template option.
  default: null
  type: str
```

Figure 2-12:

11. In `ansible-doc` leave by pressing the button **q**. You can also use the up and down arrows on your keyboard to scroll through the content.
Press **q** on your keyboard to leave the module documentation.
12. From the Jumpshot desktop, double-click the **Chrome** icon to open the Chrome browser.

13. In the Chrome browser, click the **NetApp Ansible Modules** bookmark in the bookmarks bar.

This page has a list of links to documentation for all of the netapp.ontap modules. The documentation is very similar to the documentation available with the ansible-doc command. As you use NetApp modules throughout this lab, consider referring to this documentation to learn more about each module.

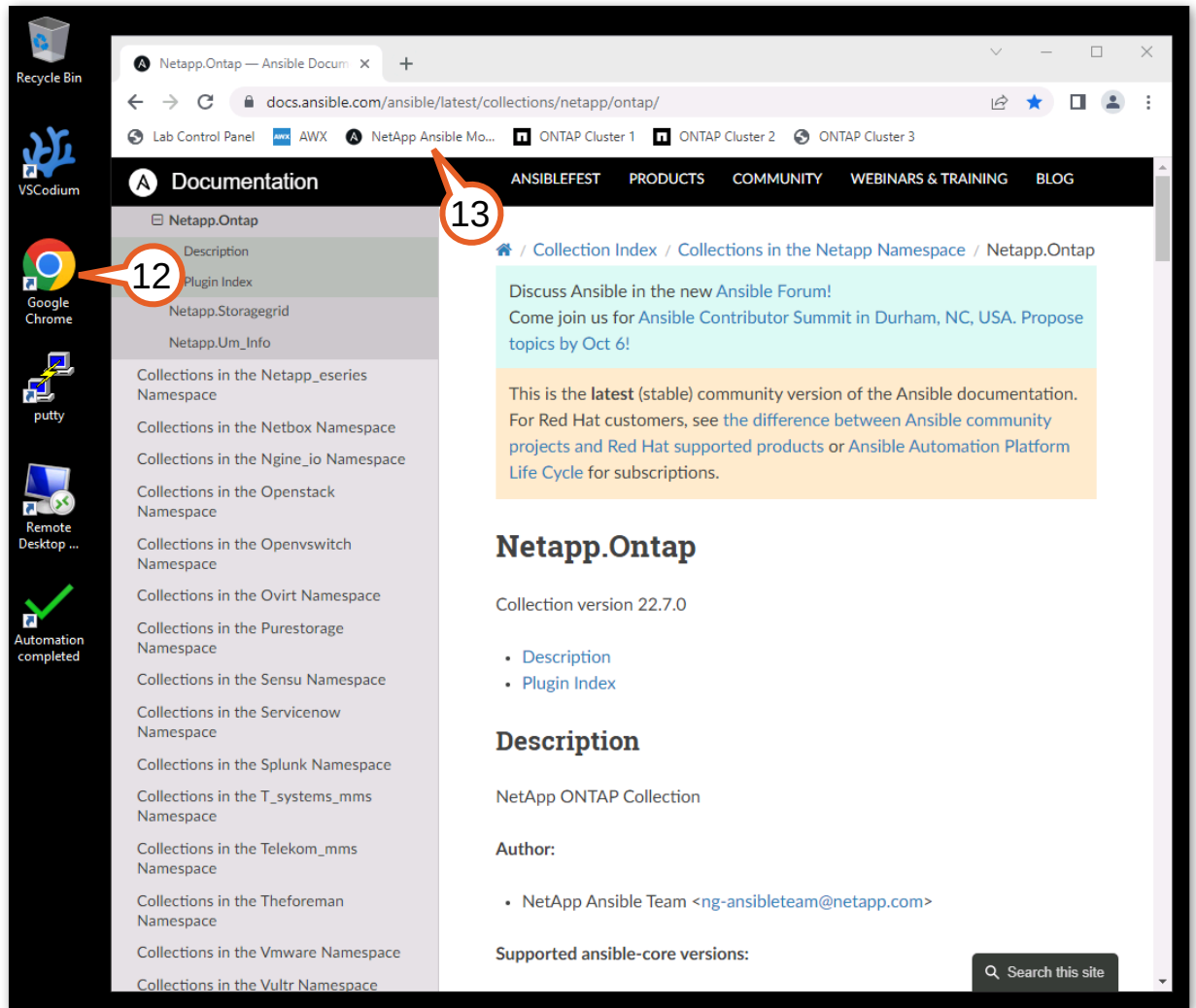


Figure 2-13:

Takeaway: Ansible modules and their documentation make it easy to perform tasks on managed nodes from your Ansible control node.

In the next activity, you will take a look at a NetApp module while writing your first Ansible playbook.

2.1.3 Write an Ansible Playbook Using an ONTAP Module

While Ansible ad-hoc commands are useful for simple, less frequent operations, you will likely find Ansible most useful for testing and predictably repeating more complex sets of tasks. Ansible playbooks are designed for this purpose.

An Ansible playbook is a YAML file that describes a set of tasks to configure a system. It's the primary way that Ansible delivers automation, and it's where you describe the desired state of your system using Ansible's modules.

A playbook can have multiple plays, each targeting a different set of hosts and containing a series of tasks to be executed in order.

Here's the basic structure of an Ansible playbook:

```
---
- name: Play 1
  hosts: target_hosts
  tasks:
    - name: Task 1
      module_name:
        key: value
    - name: Task 2
      second_module_name:
        key: value
- name: Play 2
  hosts: other_target_hosts
  tasks:
    - name: Task 3
      third_module_name:
        key: value
...
```

In this playbook you can see two plays: **Play 1** and **Play 2**. Each play has a common structure with the following elements:

- **name:** Describes the play or task.
- **hosts:** Specifies which host or group of hosts the play should run on.
- **tasks:** Lists the tasks to be executed. Each task calls an Ansible module, which is a piece of code that performs a specific action.
- **module_name, another_module_name, and third_module_name:** The names of the Ansible modules to use, such as ping, command or file from the previous activity.
- **key: value:** The parameters passed to the module.



Note: Notice that the playbook begins with `---` and ends with `...`, which are YAML document markers. Also it is important to understand that YAML files are indented with spaces and not tabs.

The ordering of the contents within a Playbook is important, because Ansible executes plays and tasks in the order they are presented.

Additionally, an Ansible playbook should be idempotent, meaning it produces the same results no matter how many times it is performed. So if a Playbook is run once to put the hosts in the correct state, it should be safe to run it a second or third time with no further changes made to the hosts. Most Ansible modules are idempotent, so it is usually easy to satisfy this objective.

In this activity, you examine a simple playbook that gathers and displays information about an ONTAP system, and through this you will learn how to write and execute simple playbooks.

1. Return to the VSCode window, and hover your cursor over the Ansible Workspace heading in the Explorer panel. Click the **New File icon** from the icons that appear.

2. A text prompt appears where you can enter the new file's name.
Enter **playbook.yml** in the field and then press **enter** on your keyboard.

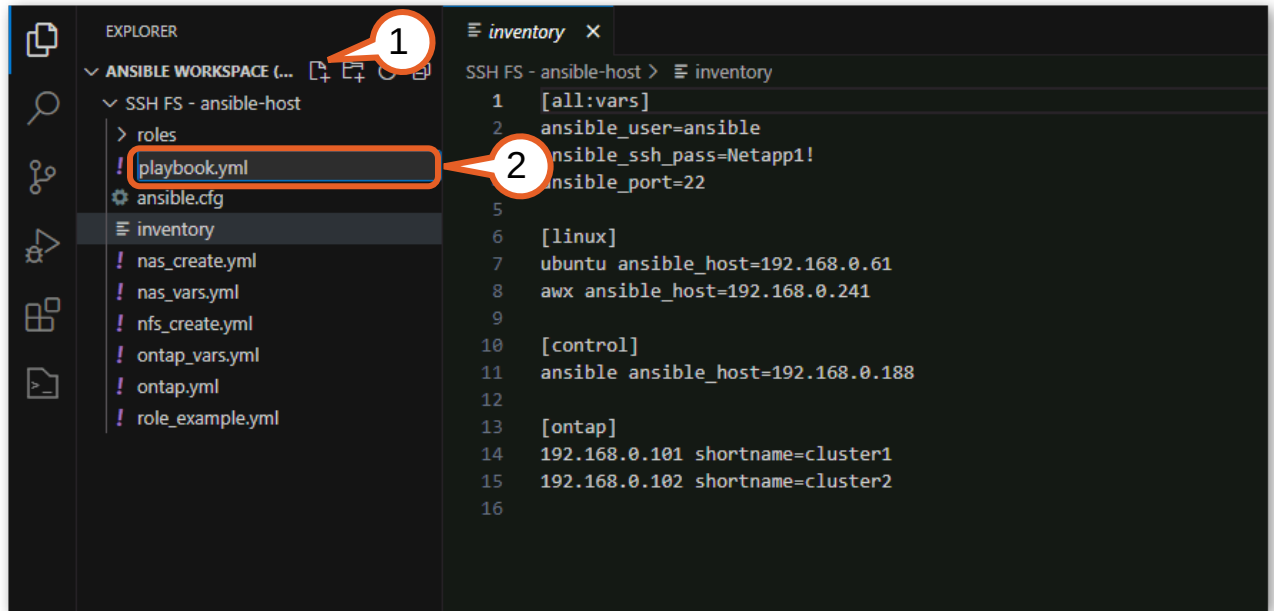


Figure 2-14:

3. The file opens in a tab on the right.

Enter this text into the file:

```
---
- hosts: localhost
  gather_facts: false
  name: Connectivity Test & Display ONTAP Info
```



Tip: If attempting to copy/paste from this guide, ensure that the cursor in VSCode is all the way to the left margin on a new line. VSCode auto-indents new lines which can cause the indentation to be incorrect when pasting. Also remember that Ansible playbooks are indented with spaces, not tabs.

4. Notice that the playbook contains a single play with the name “Connectivity Test & Display ONTAP Info.” Despite the fact that this play’s purpose is to gather info from an ONTAP cluster, the play starts with hosts keyword set to “localhost”. This is because NetApp Ansible modules work by making ONTAP REST API calls (or ONTAPI calls for older versions of ONTAP). This means that the Ansible modules are run locally on the control node rather than directly on the ONTAP system.
5. Also at the top of the play is the “gather_facts” keyword. This is a setting that controls if and how Ansible gathers facts about the specified host. Facts are details about the system, like network interfaces, operating system, and IP

addresses. By default, `gather_facts` is set to `true`, but setting it to `false` in this case saves time since our tasks will be making API calls from our control node.

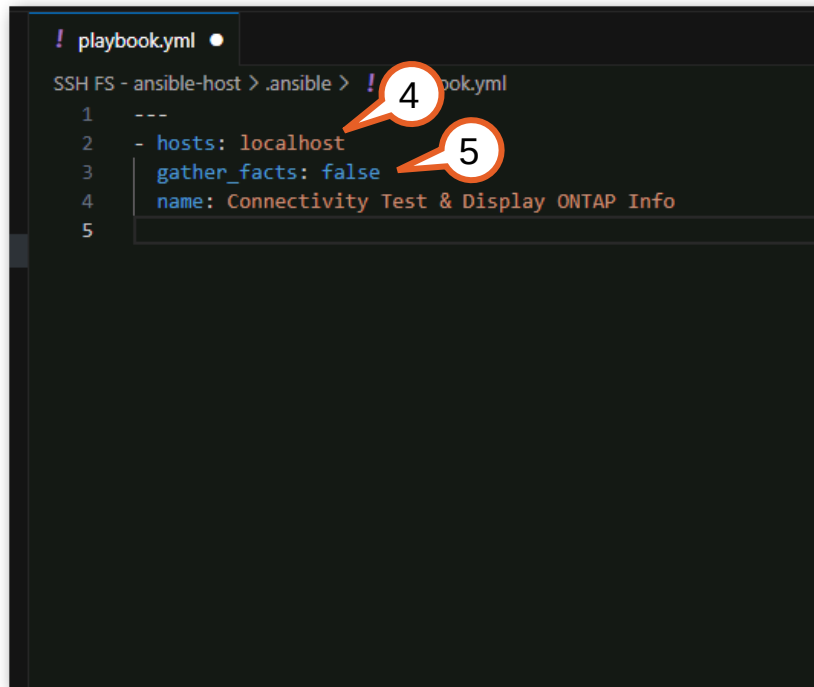


Figure 2-15:

6. Next is the list of tasks.

Add this tasks heading along with the first task beneath the name line.

```
tasks:
- name: "Collect ONTAP info"
  netapp.ontap.na_ontap_rest_info:
    hostname: cluster1.demo.netapp.com
    username: admin
    password: Netapp1!
    https: true
    validate_certs: false
    gather_subset: "svm/svms"
    register: info
```

7. The first task is called “Collect ONTAP info” and uses the `na_ontap_rest_info` module.



Note: Notice that the collection is specified in the module name with a `netapp.ontap.` prefix. You will explore a way to simplify this in the next activity.

Below the module name is a list of key/value pairs that define the module’s parameters. According to the documentation the only required parameter for this module is `hostname`, however in this case these extra

parameters are needed for the module to execute successfully. Here is a brief description of each of the parameters and what they do:

- **hostname:** to identify ONTAP cluster to run this against.
- **username:** define the username to run this operation as.
- **password:** define the password for that username.
- **https:** configure the task to query the API over https instead of http.
- **validate_certs:** disable certificate validation since we are using a self-signed certificate.
- **gather_subset:** restricts the information collected to a given REST API path, in this case the “svm/svms” REST API path is chosen so that only Storage Virtual Machine (SVM) information is returned.



Tip: The best way to determine which parameters to use and which values to input is by using the module documentation. You can use the [ansible-doc](#) command like you saw in the previous activity, or you can use the web documentation that you looked at in the previous activity.

8. At the bottom of the task is an Ansible directive called “register” which will allow you to print the output of the `na_ontap_rest_info` module. Ansible doesn't automatically print the output of tasks, so the output of this module will not be displayed without using register, which captures the output and assigns it to a variable.

The value you enter after the directive is “info”, and this will be the name of the variable that contains the output.

Figure 2-16:

9. Add this second task at the bottom of the file and ensure that the indentation of this task matches the first task (starting with the - name statement):

```
- name: Print ONTAP Response
  debug:
    msg: "{{ info }}"
```

10. The second task in the play “Print ONTAP Response” uses the debug module to print the output you captured from the previous task.

The `msg` parameter for the debug module is a string that includes the `info` variable created by the register directive in the prior task. Notice that in Ansible, variables are inserted within a string using two pairs of curly braces surrounding the variable name. You will explore variables in greater detail in the next activity.

11. Now save your changes by clicking the **save icon** near the top-right of the window.

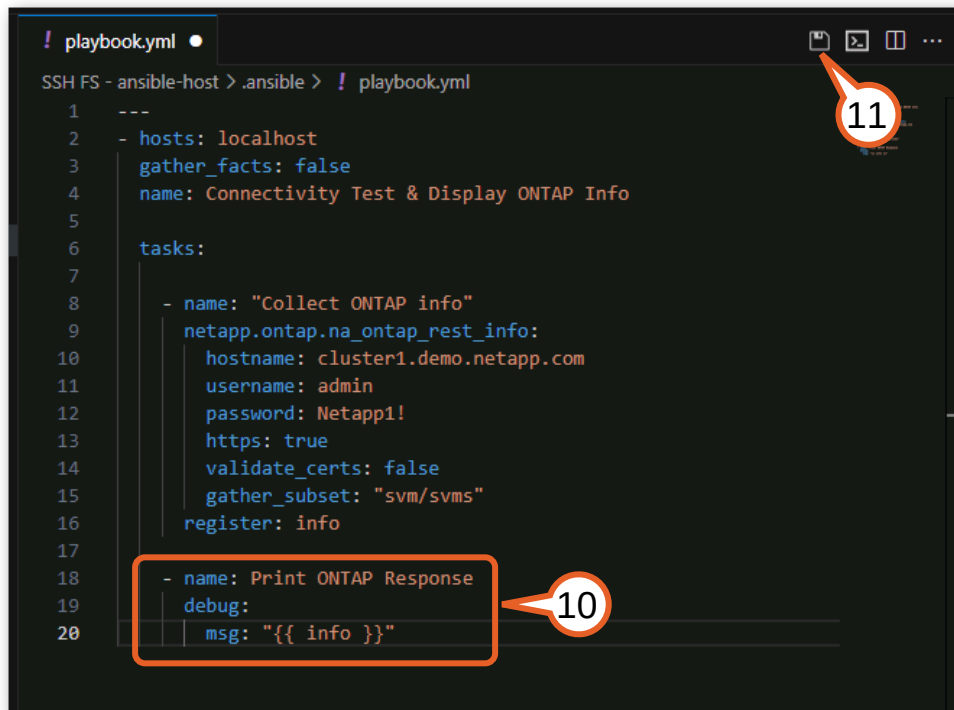


Figure 2-17:

12. The home directory for the ansible user contains a playbook called `ontap.yml` that should be identical to the playbook that you just created in `playbook.yml`. In the terminal pane, run this playbook by issuing this command:

```
ansible-playbook ontap.yml
```



Tip: The playbook file “ontap.yml” is the expected result of the steps you took in this activity. Running this playbook file in place of your `playbook.yml` file ensures that there are no syntax issues preventing you from continuing on in the lab. If you would like, you can run your playbook file instead, or you can use the `ontap.yml` file to check your work.

13. Notice the output shows the start of the play, followed by both tasks in the play with their name, followed by the result, which is the status “ok”.
14. For the second task, the debug message that includes the registered output of the first task is printed. The output includes the names and UUIDs of the two SVMs hosted on the ONTAP cluster.

15. At the bottom is the PLAY RECAP, which shows you a breakdown of the hosts acted upon by the playbook, and totals of the various outcomes of each task. In this case there is a count of 2 for the “ok” status.

```

ansible@ansible:~$ ansible-playbook ontap.yml

PLAY [Connectivity Test & Display ONTAP Info] *****

TASK [Collect ONTAP info] **
ok: [localhost]

TASK [Print ONTAP Response]
ok: [localhost] =>
  msg:
    changed: false
    failed: false
    ontap_info:
      svm/svms:
        _links:
          self:
            href: /api/svm/svms?max_records=1024&fields=
            num_records: 2
            records:
              - _links:
                  self:
                    href: /api/svm/svms/7bd95f96-ee7c-11ed-ac8c-005056b03f7b
                    name: svm2_cluster1
                    uuid: 7bd95f96-ee7c-11ed-ac8c-005056b03f7b
              - _links:
                  self:
                    href: /api/svm/svms/c047fed8-ee7b-11ed-ac8c-005056b03f7b
                    name: svm1_cluster1
                    uuid: c047fed8-ee7b-11ed-ac8c-005056b03f7b

PLAY RECAP *****
localhost                : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ansible@ansible:~$
  
```

Figure 2-18:

Takeaway: Playbooks allow you to execute multiple tasks against a set of hosts, and these tasks can be configured to influence one another using Ansible features like “register”.

Now that you have a basic playbook using a `netapp.ontap` module, in the next section of the lab, you will begin to incorporate more advanced Ansible’s features into your playbook, as well as additional `netapp.ontap` modules.

2.2 Write a Playbook to Provision ONTAP Storage

The previous activities showed you the basics of working with Ansible. In this section of the guide, you are going to evolve your simple playbook into an NAS provisioning playbook, and while doing so learn some more advanced Ansible skills that will add new capabilities and flexibility to your automated workflows.

In this section you perform the following tasks:

Table 2:

Activities
Use Playbook Variables to Simplify Tasks
Add NFS Provisioning Tasks

2.2.1 Use Playbook Variables to Simplify Tasks

Ansible exists to make tasks simple and repeatable. However, not all systems are exactly alike and often require some change to the way an Ansible playbook is run. Variables provide a convenient way to change specific details each time a playbook is run.

Ansible substitutes a variable with its value when a task is executed. You can define the value of variables in a number of ways, and each location has a clear precedence. Here are some common ways to define a variable, listed in order of precedence where the last example overrides all others:

- **Inventory file:** variables can be associated with specific hosts in your Ansible inventory.
- **Within the playbook:** you can list variables and their values within a playbook using “vars:”.
- **Variable files:** you can create a file containing a list of variables and their values, and then reference this file in your playbook using “var_files”.
- **Command line:** you can pass variables when running a playbook using “--extra-vars”, followed by “key-value” pairs or a JSON.

You saw a brief example of a variable in the previous activity (when you used “register” to define a variable), but as a reminder, variables are referenced in playbooks by placing the variable name in quoted double curly braces:

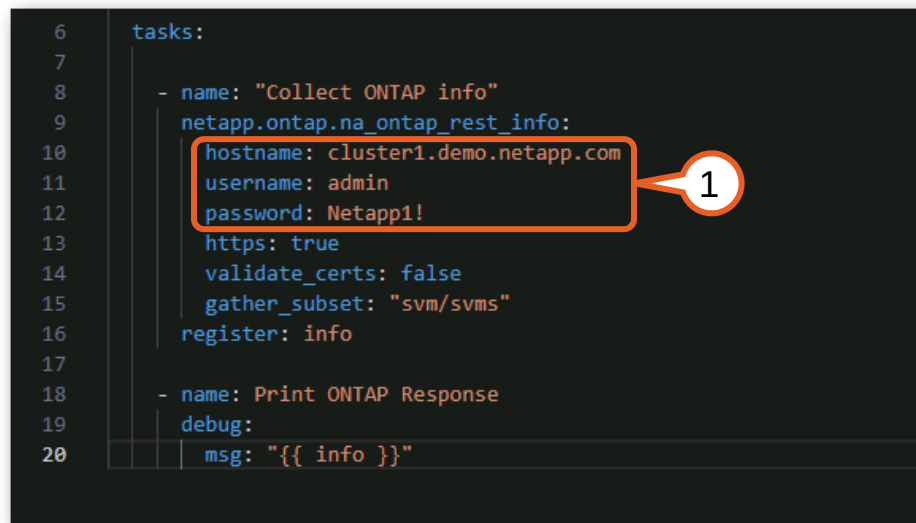
```
some_parameter: "{{ variable_name }}"
```

Because your goal is to develop an ONTAP storage provisioning playbook, it is a good idea to start incorporating variables now. ONTAP modules tend to require similar parameter values for each task (for things like selecting the desired ONTAP resources and authenticating) so having those variables set as we add tasks to the playbook in the next activity will simplify development.

In this activity you modify your ONTAP playbook to incorporate variables. In addition, you also take advantage of some related playbook features like “vars_prompt” to avoid storing plain text passwords, and module_defaults to make your ONTAP playbook more concise.

1. You should still have VSCodium open to the “playbook.yml” file. Locate the “netapp.ontap.na_ontap_rest_info” task in your playbook.

The hostname, username, and password parameters are things you might change when reusing a provisioning playbook. By changing these values to variables, you make it easy to run the playbook against different ONTAP clusters or using different user accounts.



```
6 tasks:
7
8   - name: "Collect ONTAP info"
9     netapp.ontap.na_ontap_rest_info:
10       hostname: cluster1.demo.netapp.com
11       username: admin
12       password: Netapp1!
13       https: true
14       validate_certs: false
15       gather_subset: "svm/svms"
16       register: info
17
18   - name: Print ONTAP Response
19     debug:
20       msg: "{{ info }}"
```

Figure 2-19:

2. Replace the values in these three lines with variable names like this:

```
hostname: "{{ netapp_hostname }}"
username: "{{ netapp_username }}"
```

```
password: "{{ netapp_password }}"
```



Tip: Remember if you are copying and pasting text from the guide, ensure that the cursor in VSCodium is all the way to the left margin on a new line. VSCodium auto-indents new lines which can cause the indentation to be incorrect when pasting.

```
5
6 tasks:
7
8   - name: "Collect ONTAP info"
9     netapp.ontap.na_ontap_rest_info:
10      hostname: "{{ netapp_hostname }}"
11      username: "{{ netapp_username }}"
12      password: "{{ netapp_password }}"
13      https: true
14      validate_certs: false
15      gather_subset: "svm/svms"
16      register: info
17
18   - name: Print ONTAP Response
19     debug:
20       msg: "{{ info }}"
```

Figure 2-20:

3. With the variables names in place, now you need to define these variables so Ansible knows what values to insert for each parameter.

Above the task, on a new line beneath the play's name, insert these lines, making sure the indentation matches the screenshot below:

```
vars:
  netapp_hostname: cluster1.demo.netapp.com
  netapp_username: admin
  netapp_password: Netapp1!
```

```
1 ---
2 - hosts: localhost
3   gather_facts: false
4   name: Connectivity Test & Display ONTAP Info
5
6   vars:
7     netapp_hostname: cluster1.demo.netapp.com
8     netapp_username: admin
9     netapp_password: Netapp1!
10
11  tasks:
```

Figure 2-21:

4. You should now have a playbook that correctly uses variables. Any subsequent references to one of your variables, such as “{{ netapp_hostname }}”, will be replaced by Ansible with the value you defined under “vars”.



Tip: You will try out your edited playbook at the end of this activity, but feel free to try it on your own as you make changes, just make sure to save the playbook using the save icon at the top-right of the window.

There are more steps you can take to improve this playbook before adding provisioning tasks. One helpful way to make ONTAP playbooks more concise and readable is to use `module_defaults`. Many of the parameters for ONTAP modules are used for things common to all modules within the `netapp.ontap` collection, such as specifying the desired ONTAP resource and authenticating. Module_defaults allow you to define these parameters once for a group of modules, making your playbook much more concise.

Insert this `module_default` definition above your “vars” definition near the top of the playbook:

```
module_defaults:
  group/netapp.ontap.netapp_ontap:
    hostname: "{{ netapp_hostname }}"
    username: "{{ netapp_username }}"
    password: "{{ netapp_password }}"
    https: true
    validate_certs: false
    use_rest: auto
```

5. Notice the first line beneath “`module_defaults`” is the name of the module group. This means that the following `module_defaults` will apply to all modules in the `netapp.ontap` collection.
6. Beneath that are the same parameters used in the `netapp.ontap.na_ontap_rest_info` task below. By placing them here in the defaults, you no longer need to have them within the task itself.

```
1  ---
2  - hosts: localhost
3    gather_facts: false
4    name: Connectivity Test & Display ONTAP Info
5
6    module_defaults:
7      group/netapp.ontap.netapp_ontap:
8        hostname: "{{ netapp_hostname }}"
9        username: "{{ netapp_username }}"
10       password: "{{ netapp_password }}"
11       https: true
12       validate_certs: false
13       use_rest: auto
14
15     vars:
16       netapp_hostname: cluster1.demo.netapp.com
17       netapp_username: admin
18       netapp_password: Netapp1!
19
20     tasks:
```

Figure 2-22:

7. Delete these five parameters (hostname, username, password, https, and validate_certs) from the netapp.ontap.na_ontap_rest_info task.



Important: Do not delete the “gather_subset” task. This task is specific to this module and so is not suitable for inclusion in module_defaults.

```
18
19 tasks:
20
21   - name: "Collect ONTAP info"
22     netapp.ontap.na_ontap_rest_info:
23       hostname: cluster1.demo.netapp.com
24       username: admin
25       password: Netapp1!
26       https: true
27       validate_certs: false
28       gather_subset: "svm/svms"
29     register: info
30
31   - name: Print ONTAP Response
32     debug:
33       msg: "{{ info }}"
```

Figure 2-23:

When deleted, the task should look like this.

```
18
19 tasks:
20
21   - name: "Collect ONTAP info"
22     netapp.ontap.na_ontap_rest_info:
23       gather_subset: "svm/svms"
24     register: info
25
26   - name: Print ONTAP Response
27     debug:
28       msg: "{{ info }}"
```

Figure 2-24:

With module_defaults set up, notice that the na_ontap_rest_info module is much more concise. More importantly, your play can be more concise as you add additional tasks from the netapp.ontap collection which do not need these parameters set.

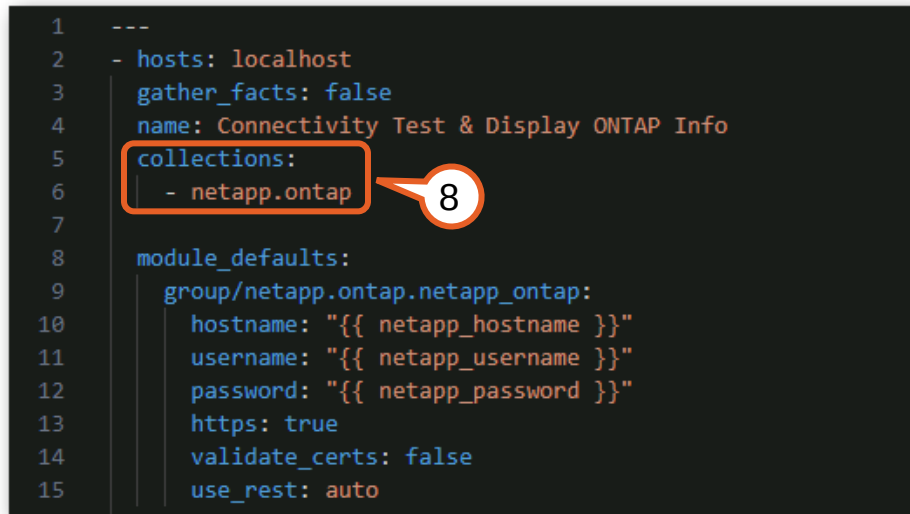


Tip: You can also use a feature of YAML called “anchors and aliases” to minimize the repetition of sets of variables. You do this by assigning an alias to a set of variables that you can then reference elsewhere in your playbook. For more information, check the Anchors and Aliases documentation in the [References](#) section.

8. Make your netapp.ontap tasks even more concise by defining the netapp.ontap module collection namespace to be included in the playbook using the “collections:” keyword.

Add these lines in a new line beneath the play’s name:

```
collections:
  - netapp.ontap
```



```
1  ---
2  - hosts: localhost
3    gather_facts: false
4    name: Connectivity Test & Display ONTAP Info
5    collections:
6      - netapp.ontap
7
8    module_defaults:
9      group/netapp.ontap.netapp_ontap:
10        hostname: "{{ netapp_hostname }}"
11        username: "{{ netapp_username }}"
12        password: "{{ netapp_password }}"
13        https: true
14        validate_certs: false
15        use_rest: auto
```

Figure 2-25:

9. With the collection namespace defined, you can delete the namespace from the module name used in your task. Change the module name from “netapp.ontap.na_ontap_rest_info” to just “na_ontap_rest_info” by deleting “netapp.ontap.”.

```
20
21 tasks:
22
23   - name: "Collect ONTAP info"
24     netapp.ontap.na_ontap_rest_info:
25       gather_subset: "svm/svms"
26       register: info
27
28   - name: Print ONTAP Response
29     debug:
30       msg: "{{ info }}"
```

Figure 2-26:

After deleting, the task module should look like this:

```
20
21 tasks:
22
23   - name: "Collect ONTAP info"
24     na_ontap_rest_info:
25       gather_subset: "svm/svms"
26       register: info
27
28   - name: Print ONTAP Response
29     debug:
30       msg: "{{ info }}"
```

Figure 2-27:

Now any additional modules from the netapp.ontap collection do not need the namespace name to be included, making the playbook more concise and readable.

10. Finally, it is a good idea to avoid keeping your password in your playbook, so instead you should use `vars_prompt` and have anyone running the playbook supply the password at runtime.

First, remove the `netapp_password` variable definition from “vars”.

```
7
8  module_defaults:
9      group/netapp.ontap.netapp_ontap:
10         hostname: "{{ netapp_hostname }}"
11         username: "{{ netapp_username }}"
12         password: "{{ netapp_password }}"
13         https: true
14         validate_certs: false
15         use_rest: auto
16
17  vars:
18     netapp_hostname: cluster1.demo.netapp.com
19     netapp_username: admin
20     netapp_password: Netapp1!
21
22  tasks:
```

Figure 2-28:

11. Now place these lines starting in a new line above your “vars:” definition:

```
vars_prompt:
- name: "netapp_password"
  prompt: "Enter the NetApp Admin Password"
  private: true
  confirm: true
```

A `Vars_prompt` with this configuration will prompt the CLI user to enter the ONTAP admin password before the playbook is run. Because “confirm” is true, the password needs to be entered twice, and because “private” is true, the input will be hidden.



Tip: Ansible also has a feature called Ansible Vault which can store passwords in encrypted files, allowing them to be safely saved. You can find out more about this feature in the documentation linked in the [References](#) section of this guide.

12. Now save your changes by clicking the **save icon** near the top-right of the window.

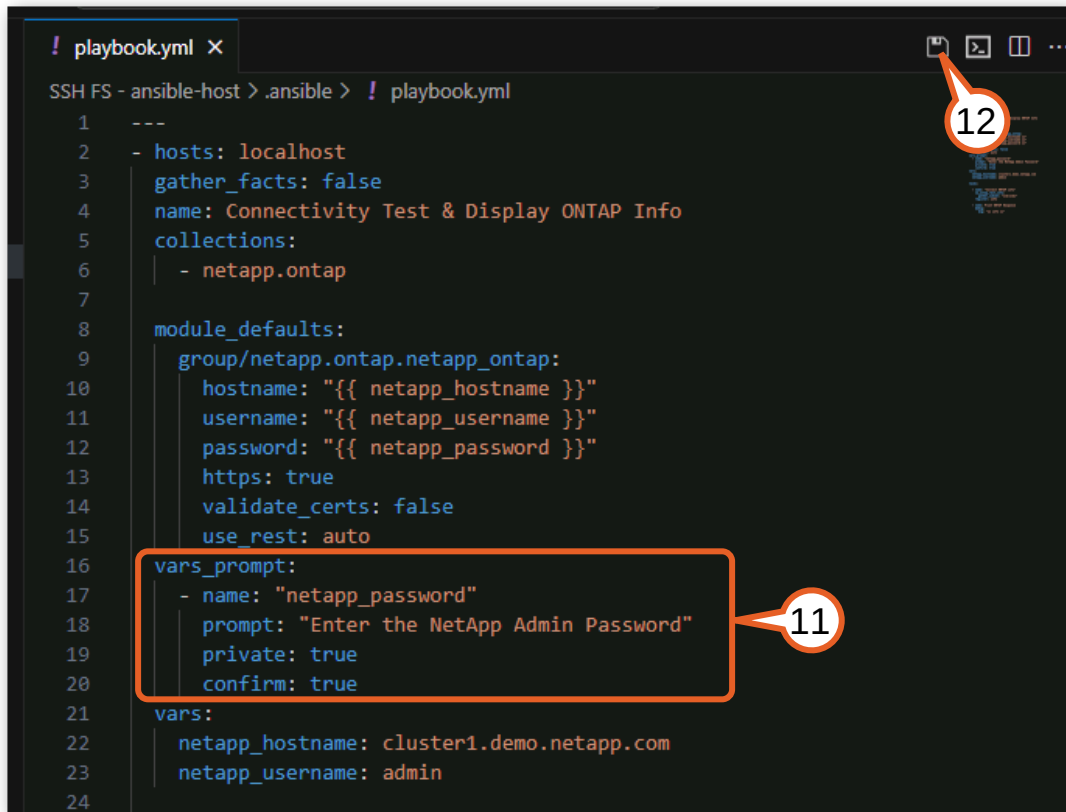


Figure 2-29:

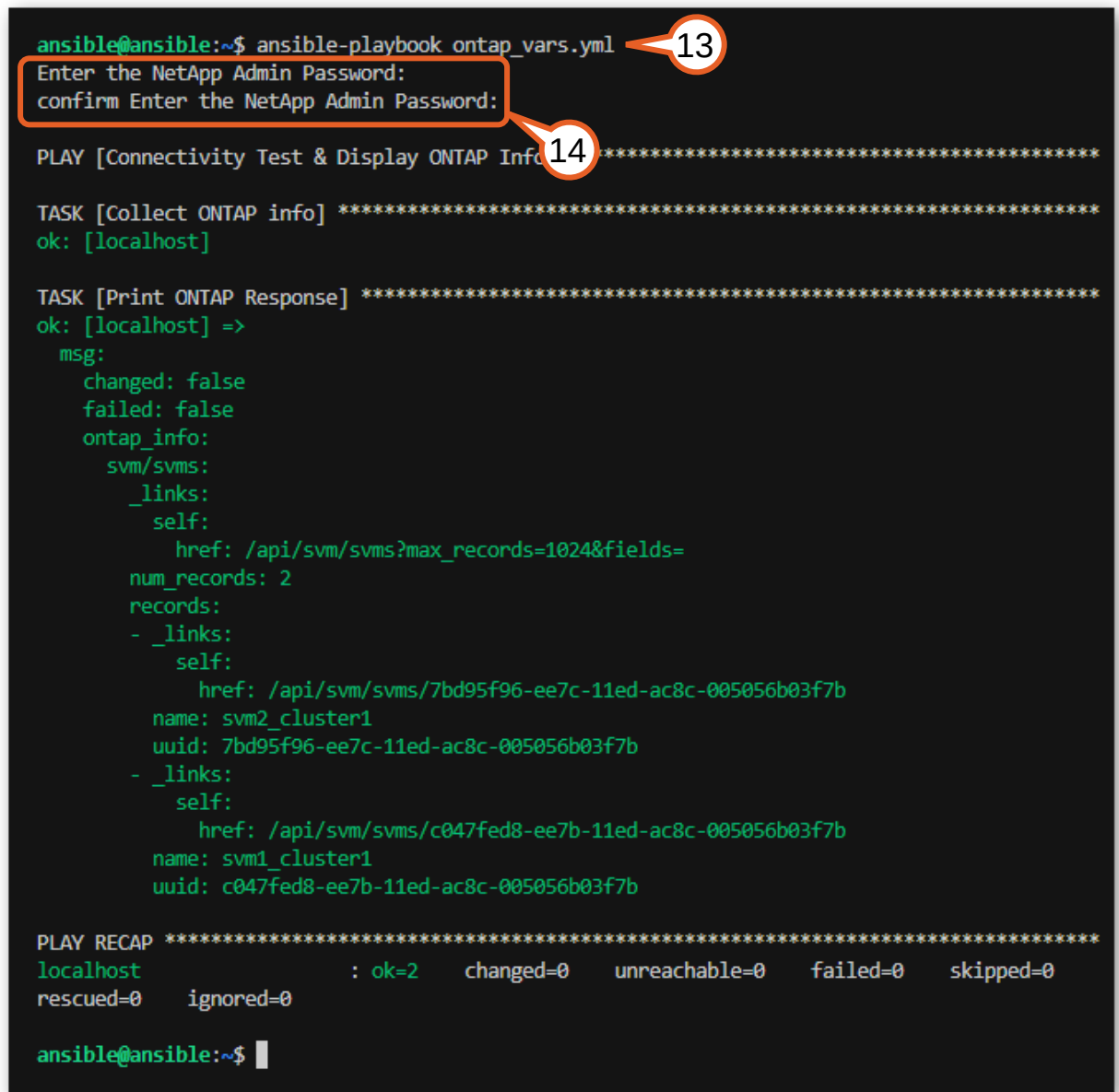
13. The home directory for the ansible user contains a playbook called `ontap_vars.yml` that should be identical to the playbook that you just edited in `playbook.yml`. In the terminal pane, run this playbook by issuing this command:

```
ansible-playbook ontap_vars.yml
```



Tip: The playbook file “`ontap_vars.yml`” is the expected result of the steps you took in this activity. Running this playbook file in the place of your `playbook.yml` file ensures that there are no syntax issues preventing you from continuing on in the lab. If you would like, you can run your `playbook` file instead, or you can use the `ontap_vars.yml` file to check your work.

14. The playbook should work just like it did before, only now prompting you for a password. When prompted for the password, enter **Netapp1!** and hit enter. You will need to do this twice.



```
ansible@ansible:~$ ansible-playbook ontap_vars.yml
Enter the NetApp Admin Password:
confirm Enter the NetApp Admin Password:

PLAY [Connectivity Test & Display ONTAP Info] *****

TASK [Collect ONTAP info] *****
ok: [localhost]

TASK [Print ONTAP Response] *****
ok: [localhost] =>
  msg:
    changed: false
    failed: false
    ontap_info:
      svm/svms:
        _links:
          self:
            href: /api/svm/svms?max_records=1024&fields=
            num_records: 2
            records:
              - _links:
                  self:
                    href: /api/svm/svms/7bd95f96-ee7c-11ed-ac8c-005056b03f7b
                    name: svm2_cluster1
                    uuid: 7bd95f96-ee7c-11ed-ac8c-005056b03f7b
              - _links:
                  self:
                    href: /api/svm/svms/c047fed8-ee7b-11ed-ac8c-005056b03f7b
                    name: svm1_cluster1
                    uuid: c047fed8-ee7b-11ed-ac8c-005056b03f7b

PLAY RECAP *****
localhost : ok=2    changed=0    unreachable=0    failed=0    skipped=0
rescued=0  ignored=0

ansible@ansible:~$
```

Figure 2-30:

Now that you have incorporated variables to make reuse easier, module_defaults to make your use of netapp.ontap modules more concise, and var_prompt to protect your passwords, you are ready to start adding more NetApp modules to your playbook.

2.2.2 Add NFS Provisioning Tasks

Multiple steps are required to create accessible NAS storage within ONTAP, so in order to create a provisioning playbook you need to ensure the playbook performs each of these tasks in the correct order, and with the correct parameters.

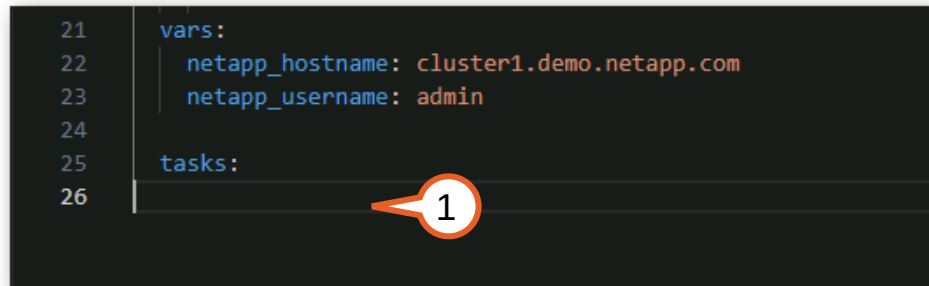
Ansible makes this easy because each task in a playbook has its own module with documented parameters, and these tasks are performed sequentially. You will start the development of this playbook with a simple sequence specific to NFS storage (you will consider CIFS in the next activity):

- Create an export policy.
- Add a rule to that policy.
- Create a volume that applies this policy.

In this activity you will add these NFS provisioning tasks to your playbook, taking advantage of the Ansible features you added in the previous activity.

1. You should still have VSCode open to your playbook file from the previous activity.

In your playbook, delete the existing two tasks by highlighting them and pressing the backspace key.



```

21 vars:
22   netapp_hostname: cluster1.demo.netapp.com
23   netapp_username: admin
24
25 tasks:
26

```

Figure 2-31:

2. Add the following task to create an export policy, using the `na_ontap_export_policy` module:

```

- name: Create Export Policy
  na_ontap_export_policy:
    state: present
    name: "{{ volname }}_policy"

```

An ONTAP export policy is a set of rules that determines which clients can access a NetApp volume or qtree over NFS and what operations they can perform.



Tip: To understand more about each module, make sure to review the module documentation. Remember there is a link to the `netapp.ontap` collection modules documentation in the in-lab browser's bookmarks bar.

3. The `state` field is an important one, indicating the desired state of the resource. As previously mentioned, playbooks are intended to describe the desired state of an environment, and so the tasks that modules perform should be idempotent. State “present” means that the desired state of the system is that this resource (in this case an export policy) should exist.

If the policy does not exist, it is created. If a policy by the same name exists, any configuration options that can change will be changed to reflect the desired state. If no changes are needed to ensure that a policy with this configuration exists, nothing will be done.



Tip: A state of “absent” will delete a resource if it exists.

4. Notice that the “name” field includes a string that has a variable and other characters. Combining one or more variables with text allows you to use variables to create naming conventions. For instance in this task a “volname” variable that has a value of vol1, would create an export policy called “vol1_policy”.

```
21 vars:
22     netapp_hostname: cluster1.demo.netapp.com
23     netapp_username: admin
24
25 tasks:
26
27     - name: Create Export Policy
28       na_ontap_export_policy:
29         state: present
30         name: "{{ volname }}_policy"
31
32
```

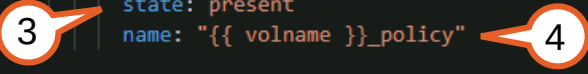


Figure 2-32:

5. You do not need to include common attributes like hostname or username because they are included with our module_defaults. However, there is another common attribute you need to add. The na_ontap_export_policy module, like all of the netapp.ontap modules you will use in this playbook requires the “vserver” parameter.

Add this line to the module_defaults for that now:

```
vserver: "{{ vserver }}"
```

6. Define the “volname” and “vserver” variables now by adding the following lines to your “vars” definition:

```
vserver: svm1_cluster1
```

```
volname: vol1
```

```
6 netapp_ontap
7
8 module_defaults:
9   group/netapp.ontap.netapp_ontap:
10     hostname: "{{ netapp_hostname }}"
11     username: "{{ netapp_username }}"
12     password: "{{ netapp_password }}"
13     vserver: "{{ vserver }}" 5
14     https: true
15     validate_certs: false
16     use_rest: auto
17   vars_prompt:
18     - name: "netapp_password"
19       prompt: "Enter the NetApp Admin Password"
20       private: true
21       confirm: true
22   vars:
23     netapp_hostname: cluster1.demo.netapp.com
24     netapp_username: admin
25     vserver: svm1_cluster1
26     volname: vol1 6
27
28 tasks:
29
```

Figure 2-33:

7. The export policy task will create the export policy, but now you need a task that creates an export policy rule. Export policies have individual rules that define the specific access permissions for different clients. To create a rule, add the following task that uses the `na_ontap_export_policy_rules` module:

```
- name: Set up Export Policy Rules
  na_ontap_export_policy_rule:
    state: present
    policy_name: "{{ volname }}_policy"
    client_match: "{{ client_match }}"
    ro_rule: sys
    rw_rule: sys
    super_user_security: sys
```

8. This module requires quite a few parameters. You could turn any of these into more easily configurable variables to make this playbook more useful, but for now only add “client_match”. Client_match is a parameter that specifies

the IP Addresses, Netgroups, or Domains of the clients that the rule applies to. The other parameters govern the access the matching clients have.

Add this line to “vars” to define a variable for client_match:

```
client_match: 192.168.0.0/24
```

```
22 vars:
23   netapp_hostname: cluster1.demo.netapp.com
24   netapp_username: admin
25   vservers: svm1_cluster1
26   volname: vol1
27   client_match: 192.168.0.0/24
28
29 tasks:
30
31   - name: Create Export Policy
32     na_ontap_export_policy:
33       state: present
34       name: "{{ volname }}_policy"
35
36   - name: Set up Export Policy Rules
37     na_ontap_export_policy_rule:
38       state: present
39       policy_name: "{{ volname }}_policy"
40       client_match: "{{ client_match }}"
41       ro_rule: sys
42       rw_rule: sys
43       super_user_security: sys
```

Figure 2-34:

9. Now you are ready to add the task that creates the volume. Insert these lines below the previous task:

```
- name: Create Volume
  na_ontap_volume:
    state: present
    name: "{{ volname }}"
    aggregate_name: "{{ aggr_name }}"
    size: "{{ size }}"
    size_unit: gb
    policy: "{{ volname }}_policy"
    junction_path: "/"
    space_guarantee: "none"
    volume_security_style: unix
```

This task creates a NetApp ONTAP volume with the name specified by the variable “volname”, located on the aggregate specified by “aggr_name”, with a size of “size” gigabytes. The volume is associated with the same export policy that was created in the first task using the policy parameter, and the volume is accessible at the junction path “/volname”. Lastly, these parameters configure the volume to not reserve any space (space_guarantee: “none”), and use UNIX-style permissions (volume_security_style: unix).



Tip: Notice that the “volname” variable is referenced multiple times in this task definition. This is another example of the value of using variables. Each run of the playbook will likely have a unique volume name,

and rather than having to change that in multiple locations within this one task, with the variable you can make the change in just one location.

```
28
29 tasks:
30
31 - name: Create Export Policy
32   na_ontap_export_policy:
33     state: present
34     name: "{{ volname }}_policy"
35
36 - name: Set up Export Policy Rules
37   na_ontap_export_policy_rule:
38     state: present
39     policy_name: "{{ volname }}_policy"
40     client_match: "{{ client_match }}"
41     ro_rule: sys
42     rw_rule: sys
43     super_user_security: sys
44
45 - name: Create Volume
46   na_ontap_volume:
47     state: present
48     name: "{{ volname }}"
49     aggregate_name: "{{ aggr_name }}"
50     size: "{{ size }}"
51     size_unit: gb
52     policy: "{{ volname }}_policy"
53     junction_path: "/" + "{{ volname }}"
54     space_guarantee: "none"
55     volume_security_style: unix
```

Figure 2-35:

10. There are two new variables used for this task, and they specify the aggregate name and volume size. Add these values to “vars”:

```
aggr_name: cluster1_01_SSD_1
size: 10
```

11. Finally, now that your playbook can provision storage, you should change the play name to “Provision NFS”.

12. Now save your changes by clicking the **save icon** near the top-right of the window.

```
! playbook.yml X
SSH FS - ansible-host > .ansible > ! playbook.yml

1  ---
2  - hosts: localhost
3    gather_facts: false
4    name: Provision NFS
5    collections:
6      - netapp.ontap
7
8    module_defaults:
9      group/netapp.ontap.netapp_ontap:
10         hostname: "{{ netapp_hostname }}"
11         username: "{{ netapp_username }}"
12         password: "{{ netapp_password }}"
13         vservers: "{{ vservers }}"
14         https: true
15         validate_certs: false
16         use_rest: auto
17     vars_prompt:
18       - name: "netapp_password"
19         prompt: "Enter the NetApp Admin Password"
20         private: true
21         confirm: true
22     vars:
23       netapp_hostname: cluster1.demo.netapp.com
24       netapp_username: admin
25       vservers: svm1_cluster1
26       aggr_name: cluster1_01_SSD_1
27       size: 10
28       volname: vol1
29       client_match: 192.168.0.0/24
30
```

Figure 2-36:

13. The home directory for the ansible user contains a playbook called “nfs_create.yml” that should be identical to the playbook that you just edited in this activity. In the terminal pane, run this playbook by issuing this command:

```
ansible-playbook nfs_create.yml
```



Tip: The playbook file “nfs_create.yml” is the expected result of the steps you took in this activity. Running this playbook file in the place of your playbook.yml file ensures that there are no syntax issues preventing you from continuing on in the lab. If you would like, you can run your playbook file instead, or you can use the “nfs_create.yml” file to check your work.

14. When prompted for the password, enter **Netapp1!** and press **enter**. You will need to do this twice.

15. The result should show each of the three tasks with a status of “changed”.

```
ansible@ansible:~$ ansible-playbook nfs_create.yml
Enter the NetApp Admin Password:
confirm Enter the NetApp Admin Password:

PLAY [Provision NFS Storage] *****

TASK [Create Export Policy] *****
changed: [localhost]

TASK [Set Up Export Policy Rules] *****
changed: [localhost]

TASK [Create Volume] *****
changed: [localhost]

PLAY RECAP *****
localhost : ok=3    changed=3    unreachable=0    failed=0    skipped=0
rescued=0    ignored=0

ansible@ansible:~$
```

Figure 2-37:



Tip: You can quickly confirm the results of the playbook in the terminal by looking for the volume that you created:

```
ssh -l admin cluster1.demo.netapp.com vol show | grep vol1
```

You will need to provide the password **Netapp1!**, and you may need to add the ONTAP cluster to your list of known hosts by entering **yes** and pressing the **enter** key.

You can also navigate to System Manager in the lab’s Chrome browser. There is a bookmark for Cluster 1 in the bookmarks bar. You can sign in with username “admin” and password “Netapp1!”.

16. Take a moment to confirm this playbook is idempotent by running the playbook one more time, repeating steps 13 and 14.

17. Notice that instead of changed (in yellow), the status is “ok” for each of the task, and is colored green. This means that the intended configuration was found and no changes were made.

```
ansible@ansible:~$ ansible-playbook nfs_create.yml
Enter the NetApp Admin Password:
confirm Enter the NetApp Admin Password:

PLAY [Provision NFS Storage] *****

TASK [Create Export Policy] *****
ok: [localhost]

TASK [Set Up Export Policy Rules] *****
ok: [localhost]

TASK [Create Volume] *****
ok: [localhost]

PLAY RECAP *****
localhost : ok=3    changed=0    unreachable=0    failed=0    skipped=0
rescued=0    ignored=0

ansible@ansible:~$
```

Figure 2-38:

Takeaway: By applying some of the basic Ansible concepts you have learned, with very little effort you can utilize netapp.ontap modules to provision storage in a repeatable way.

In the next activity, you will continue to iterate on this playbook by incorporating a CIFS protocol option.

2.2.3 Use Conditionals to Incorporate CIFS Into Your Playbook

You have a functioning NFS provisioning playbook, but many environments make use of more than one protocol, so adding CIFS provisioning to the playbook would make it more useful. Because NFS and CIFS provisioning require different steps, you will need to use additional Ansible features to accomplish this in a single playbook.

Ansible can use conditionals to execute tasks, or plays, when certain conditions are met. To implement a conditional, you use a “when” statement, followed by a condition to test. The condition is expressed using one of the available operators:

- == Compares two objects for equality.
- != Compares two objects for inequality.
- > true if the left hand side is greater than the right hand side.
- >= true if the left hand side is greater or equal to the right hand side.
- < true if the left hand side is lower than the right hand side.
- <= true if the left hand side is lower or equal to the right hand side.

Using this ability, you want to create a playbook that creates accessible storage using either NFS or CIFS protocols. The flow of tasks should look something like this:

- Create an export policy.
- If using NFS, add a rule to that policy.
- Create a volume that applies this policy.
- If using CIFS, create the CIFS share.

Note: To simplify this demonstration, your playbook will create an export policy regardless of protocol, though in reality this step should be skipped in the case of CIFS. This is because the volume module has a parameter for export policy, and it is not possible to attach a conditional to a single parameter. One way to resolve this would be to

use conditionals to both skip the export policy step in the case of CIFS and then use conditionals to skip one of two different volume creation tasks, one for CIFS and one for NFS.

In this activity, you introduce a new variable that specifies a desired protocol, and then you add Ansible conditionals that decide which tasks are run based on the value of that variable.

1. Before you add your new variable, move your variables to a variable file. Variable files, mentioned in a previous activity, allow you to maintain your variables in a separate file. When you separate the variable values from the playbook, you make it more convenient to make changes between playbook runs, as well as make it easier to share and collaborate on the playbook.

In the Explorer panel, hover your cursor near the Ansible Workspace heading and click the **New File icon**.

2. In the text prompt that appears, enter the file name **vars.yml** and then press **enter** on your keyboard.

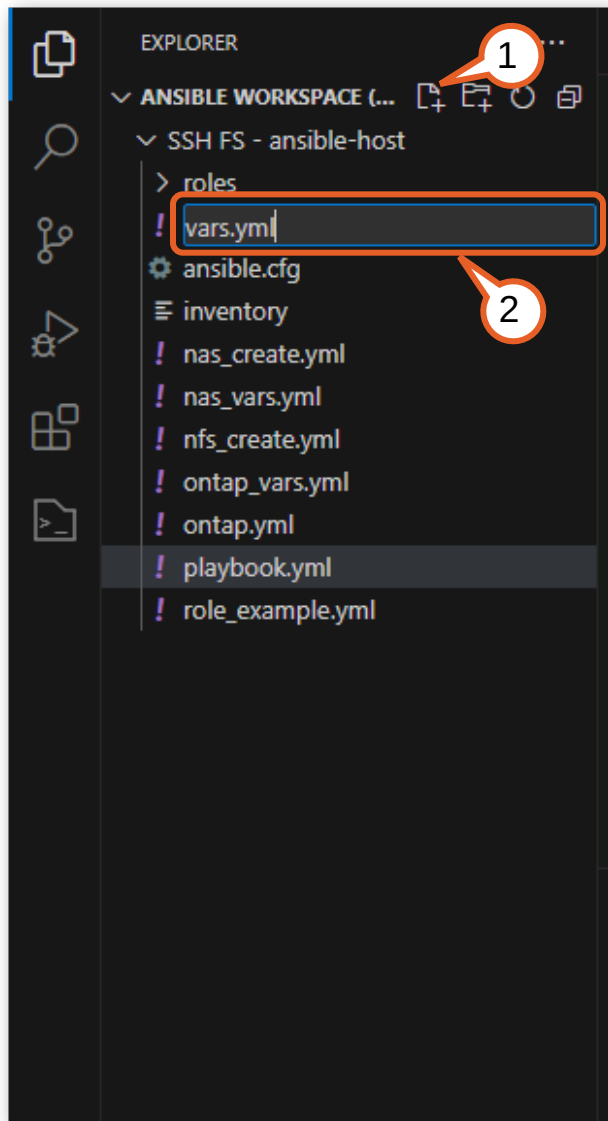


Figure 2-39:

3. The new file should open in a tab on the right.

Enter this list of variables into the file:

```
netapp_hostname: cluster1.demo.netapp.com
netapp_username: admin
vserver: svm1_cluster1
```



```
aggr_name: cluster1_01_SSD_1
volname: vol2
protocol: cifs
size: 10
client_match: 192.168.0.0/24
```

Notice in this list is your new variable “protocol” with the value of “cifs”.

4. Click the **save** icon near the top right of the window.

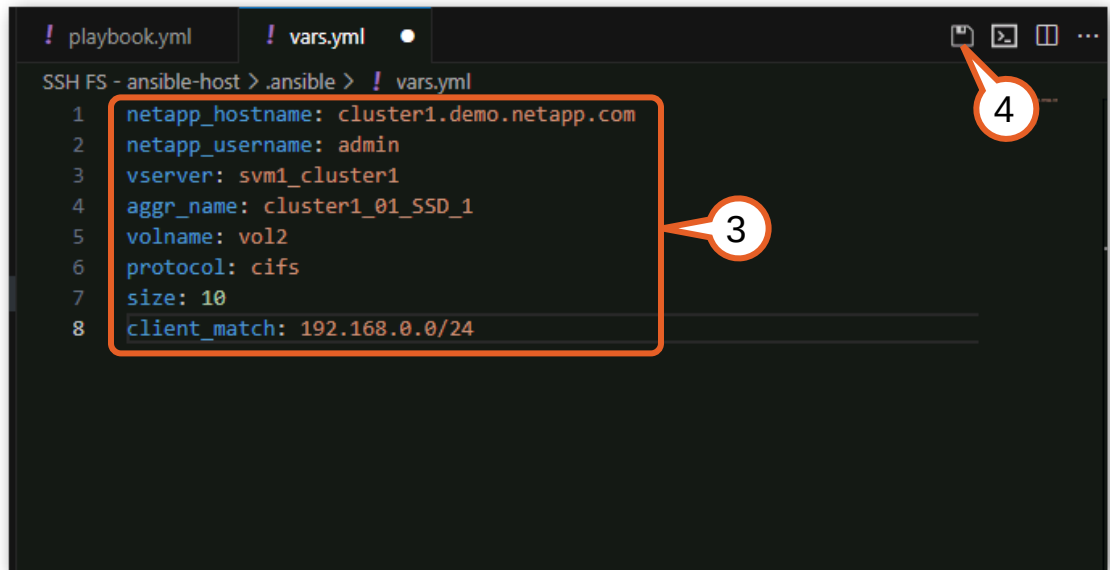


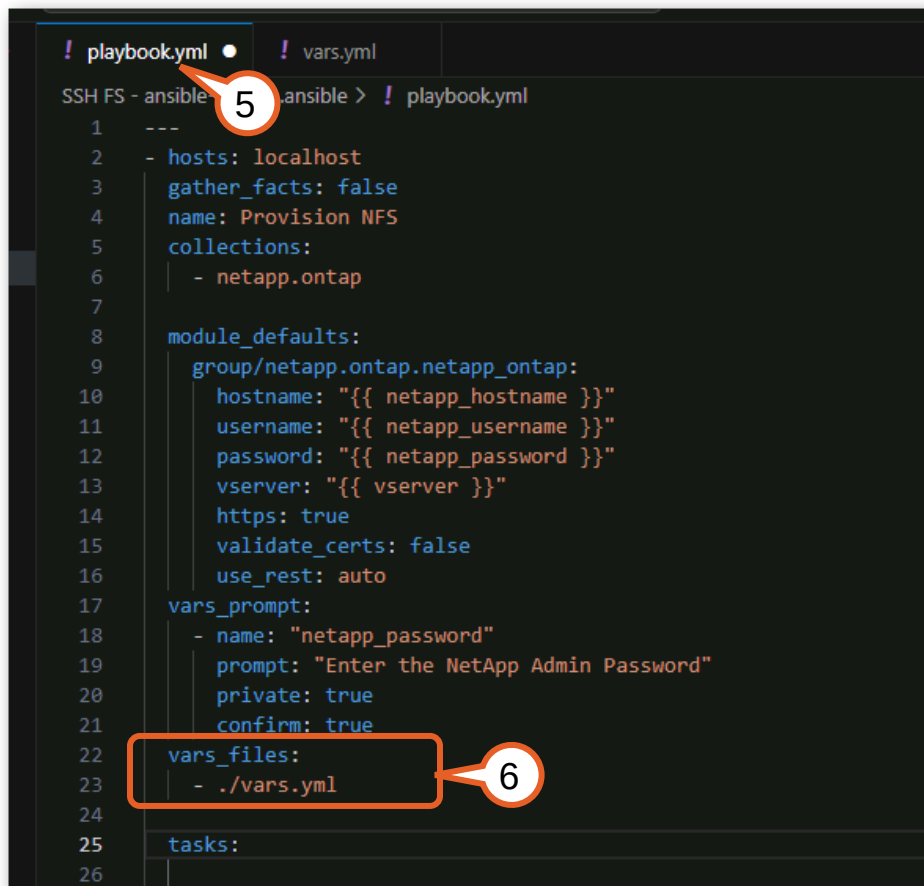
Figure 2-40:

5. Switch back to the tab for **playbook.yml**.
6. In your playbook, replace the “vars” definition with a “var_files” definition:

```
vars_files:
```

```
- ./vars.yml
```

Now your playbook will use the variable values set in the referenced file.



```
! playbook.yml • ! vars.yml
SSH FS - ansible > ! playbook.yml
1 ---
2 - hosts: localhost
3   gather_facts: false
4   name: Provision NFS
5   collections:
6     - netapp.ontap
7
8   module_defaults:
9     group/netapp.ontap.netapp_ontap:
10      hostname: "{{ netapp_hostname }}"
11      username: "{{ netapp_username }}"
12      password: "{{ netapp_password }}"
13      vserver: "{{ vserver }}"
14      https: true
15      validate_certs: false
16      use_rest: auto
17
18   vars_prompt:
19     - name: "netapp_password"
20       prompt: "Enter the NetApp Admin Password"
21       private: true
22       confirm: true
23   vars_files:
24     - ./vars.yml
25
26   tasks:
```

Figure 2-41:

- Find the Set Up Export Policy Rules task. At the bottom of the task, insert the following line, ensuring that the indentation of the line matches the module name near the top of the task:

```
when: protocol.lower() == 'nfs'
```

This “when” statement looks at the value of the protocol variable and compares it to the string “nfs”. If those values match (they are compared using the “==” operator) then this task is performed. Otherwise, the task is skipped.



Note: The '.lower()' converts the protocol variable into all lowercase to ensure matching is case insensitive.

```
24
25 tasks:
26
27   - name: Create Export Policy
28     na_ontap_export_policy:
29       state: present
30       name: "{{ volname }}_policy"
31
32   - name: Set up Export Policy Rules
33     na_ontap_export_policy_rule:
34       state: present
35       policy_name: "{{ volname }}_policy"
36       client_match: "{{ client_match }}"
37       ro_rule: sys
38       rw_rule: sys
39       super_user_security:
40       when: protocol.lower() == 'nfs'
41
```

Figure 2-42:

- Now you will add a task that creates a CIFS share in the case of the protocol variable having the value “cifs”. You want this task to follow after the volume creation because it requires an existing junction path.

Add this task beneath the na_ontap_volume task:

```
- name: Create CIFS share
  na_ontap_cifs:
    state: present
    share_name: "{{ volname }}"
    path: "{{ volname }}"
```

```
when: protocol.lower() == 'cifs'
```

Notice that the when conditional is used and ensures that the protocol variable is “cifs” before executing this task.

```
41
42   - name: Create Volume
43     na_ontap_volume:
44       state: present
45       name: "{{ volname }}"
46       aggregate_name: "{{ aggr_name }}"
47       size: "{{ size }}"
48       size_unit: gb
49       policy: "{{ volname }}_policy"
50       junction_path: "/{{ volname }}"
51       space_guarantee: "none"
52       volume_security_style: unix
53
54   - name: Create CIFS share
55     na_ontap_cifs:
56       state: present
57       share_name: "{{ volname }}"
58       path: "/{{ volname }}"
59       when: protocol.lower() == 'cifs'
```

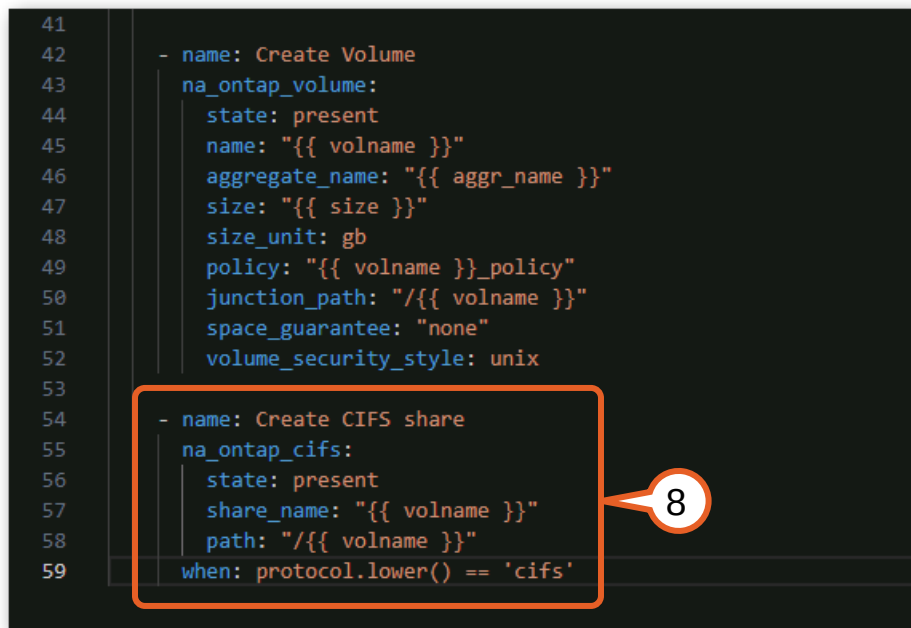
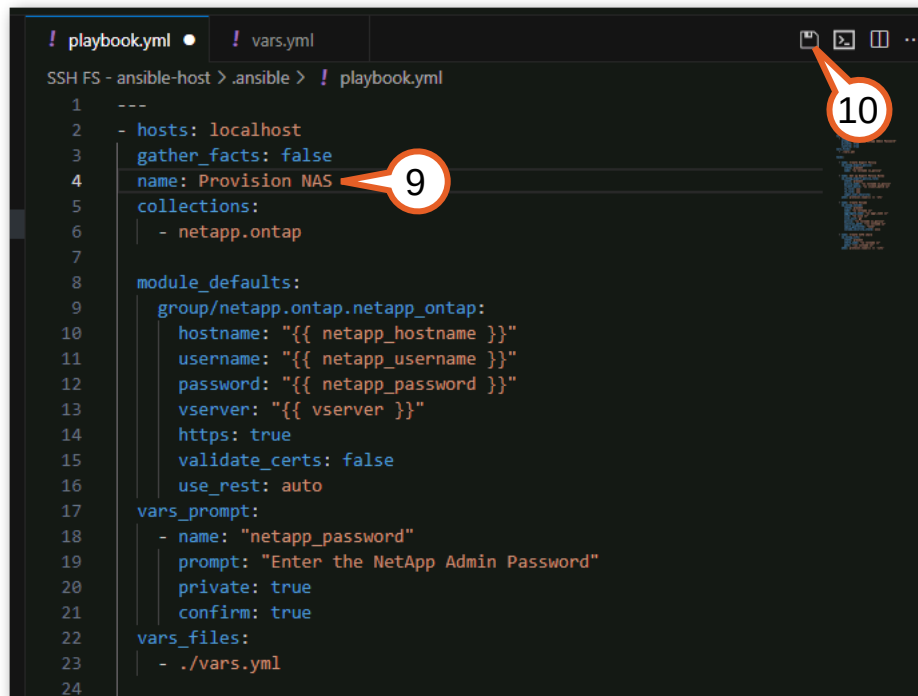


Figure 2-43:

9. Finally, now that your playbook can provision multiple protocols, you should change the play name to “Provision NAS”.
10. Now save your changes by clicking the **save** icon near the top-right of the window.



```
! playbook.yml • ! vars.yml
SSH FS - ansible-host > .ansible > ! playbook.yml
1  ---
2  - hosts: localhost
3    gather_facts: false
4    name: Provision NAS
5    collections:
6      - netapp.ontap
7
8    module_defaults:
9      group/netapp.ontap.netapp_ontap:
10        hostname: "{{ netapp_hostname }}"
11        username: "{{ netapp_username }}"
12        password: "{{ netapp_password }}"
13        vservers: "{{ vservers }}"
14        https: true
15        validate_certs: false
16        use_rest: auto
17    vars_prompt:
18      - name: "netapp_password"
19        prompt: "Enter the NetApp Admin Password"
20        private: true
21        confirm: true
22    vars_files:
23      - ./vars.yml
24
```

Figure 2-44:

11. The home directory for the ansible user contains a playbook called “nas_create.yml” that should be identical to the playbook that you just edited in this activity. In the terminal pane, run this playbook by issuing this command:

```
ansible-playbook nas_create.yml
```



Tip: The playbook file “nas_create.yml” is the expected result of the steps you took in this activity. Running this playbook file in the place of your playbook.yml file ensures that there are no syntax issues preventing you from continuing on in the lab. If you would like, you can run your playbook file instead, or you can use the nas_create.yml and nas_vars.yml files to check your work.

12. When prompted for the password, enter **Netapp1!** and press **enter**. You will need to do this twice.
13. The result should show most of the tasks with a status of “changed”, however, the Set Up Export Policy Rules task should have a status of “skipping” (in blue).



Note: You can safely ignore the warning listed under the Create Volume task.

```
ansible@ansible:~$ ansible-playbook nas_create.yml
Enter the NetApp Admin Password:
confirm Enter the NetApp Admin Password:

PLAY [Provision NAS Storage] *****

TASK [Create Export Policy] *****
changed: [localhost]

TASK [Set Up Export Policy Rules] *****
skipping: [localhost]

TASK [Create Volume] *****
[WARNING]: Cannot perform actions: modify when volume is offline.
changed: [localhost]

TASK [Create CIFS Share] *****
changed: [localhost]

PLAY RECAP *****
localhost                : ok=3    changed=3    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0

ansible@ansible:~$
```

Figure 2-45:



Tip: There are other methods for developing more complex flows within a playbook. Here are two key Ansible features for this purpose:

- **Handlers:** Handlers allow you to run a task only when a change is made in another task. For example, you may want to restart a service if a task updates the configuration of that service, but not if no change was made. Tasks can notify a handler, and when notified, a handler will execute its task.
- **Loops:** Loops make it easy to execute a single task multiple times. Examples of commonly-used loops include changing ownership on several files, creating multiple users, and repeating a polling step until a certain result is reached.

For more information about Handlers and Loops, check the documentation in the [References](#) section.

You now have a versatile playbook that can quickly and easily provision storage to your specifications. While you could stop here, Ansible has additional tools to make playbooks easier to integrate with other workflows. You will explore these in the next activity.

2.3 Make Playbooks More Useful with Ansible Roles

While a monolithic playbook can be useful on its own, repurposing individual tasks or incorporating the playbook into a larger context could be difficult. Ansible roles present an opportunity to make the storage provisioning playbook more useful and extensible.

Roles let you automatically load related vars, files, tasks, handlers, and other Ansible elements based on a known file structure. This packaging allows you to more easily reuse your Ansible content and share it with other users.

To create a role you create a specific directory structure within a “roles” directory. Each subdirectory of “roles” is the name of a role that can be referenced by a playbook. The structure must contain at least one of the expected sub-directories. Here are some common examples:

- **roles/**
 - **role_name/**
 - **tasks/**
 - main.yml
 - **vars/**
 - main.yml
 - **handlers/**
 - main.yml
 - **files/**
 - main.yml

Notice that each of these directories includes a main.yml file. This is an expected file that contains the content of the role. For example, tasks/main.yml is expected to contain the role’s tasks.

In this lab activity, you examine and use a role that is converted from the playbook you have developed so far.

1. In the menu on the left, expand the **roles** directory by clicking on it.
2. Notice that the roles directory has a “provision_nas” subdirectory. This sets the name of your role.

3. Your role has two directories, tasks and vars.
Click **tasks** > **main.yml** to open the tasks file.

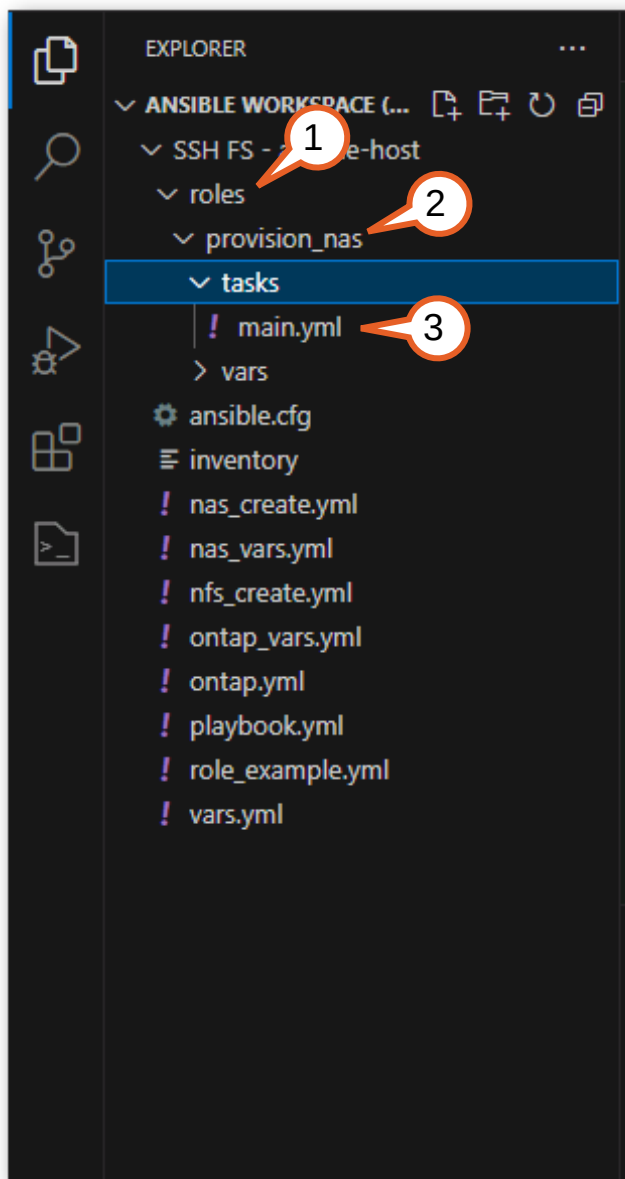
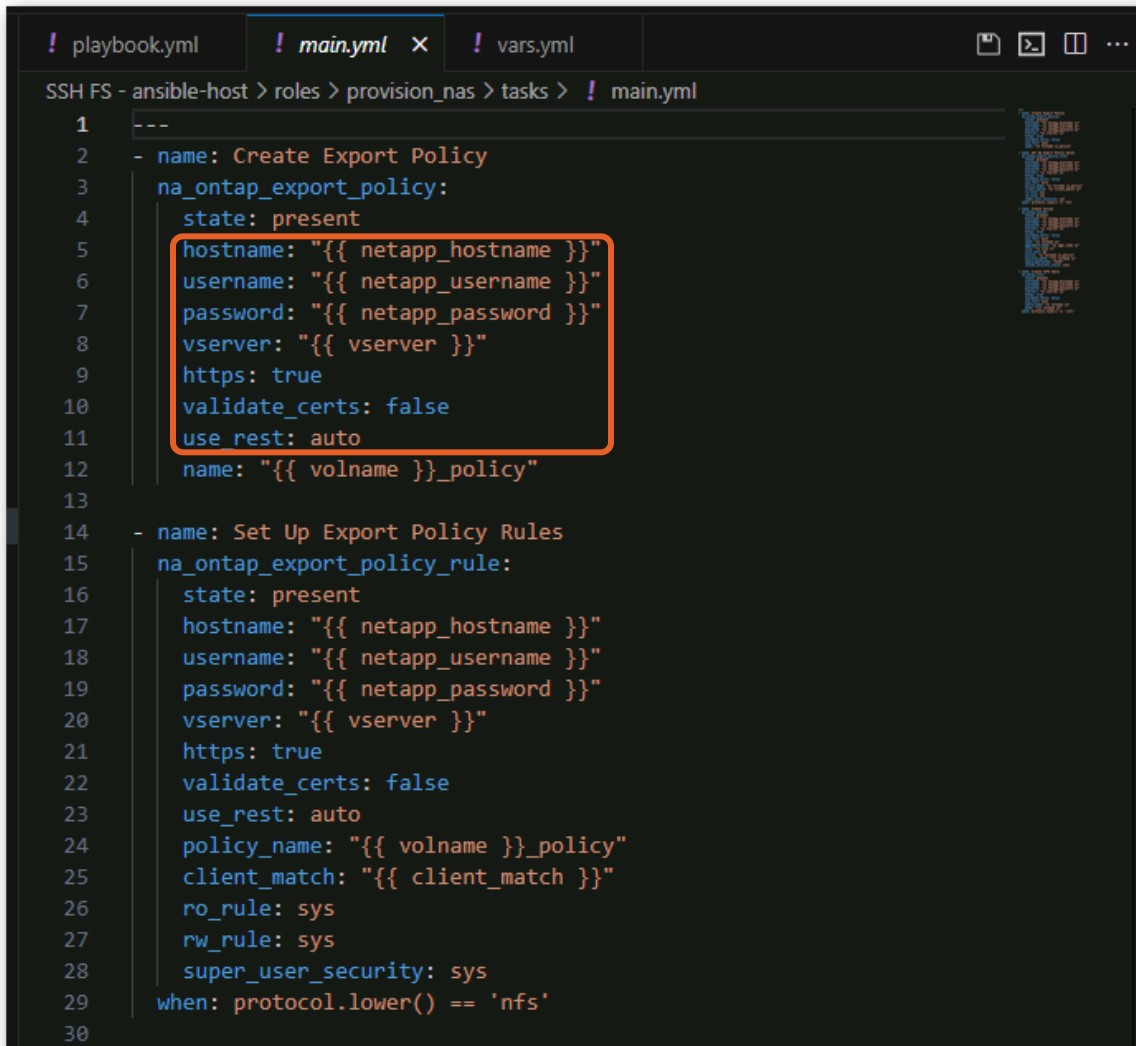


Figure 2-46:

4. The main.yml file should open in a tab on the right.

This is a list of tasks very similar to those that you had in your provisioning playbook, but there is one significant difference. Notice that the parameters that you moved to the module_defaults are present in the each of the tasks. This is because roles do not support module_defaults.



```
SSH FS - ansible-host > roles > provision_nas > tasks > ! main.yml
1  ---
2  - name: Create Export Policy
3    na_ontap_export_policy:
4      state: present
5      hostname: "{{ netapp_hostname }}"
6      username: "{{ netapp_username }}"
7      password: "{{ netapp_password }}"
8      vservers: "{{ vservers }}"
9      https: true
10     validate_certs: false
11     use_rest: auto
12     name: "{{ volname }}_policy"
13
14  - name: Set Up Export Policy Rules
15    na_ontap_export_policy_rule:
16      state: present
17      hostname: "{{ netapp_hostname }}"
18      username: "{{ netapp_username }}"
19      password: "{{ netapp_password }}"
20      vservers: "{{ vservers }}"
21      https: true
22      validate_certs: false
23      use_rest: auto
24      policy_name: "{{ volname }}_policy"
25      client_match: "{{ client_match }}"
26      ro_rule: sys
27      rw_rule: sys
28      super_user_security: sys
29      when: protocol.lower() == 'nfs'
30
```

Figure 2-47:

5. Now navigate to vars > main.yml from the panel on the left.

6. The file should open in a tab on the right.

This file is identical to the variables file that you used in your playbook.

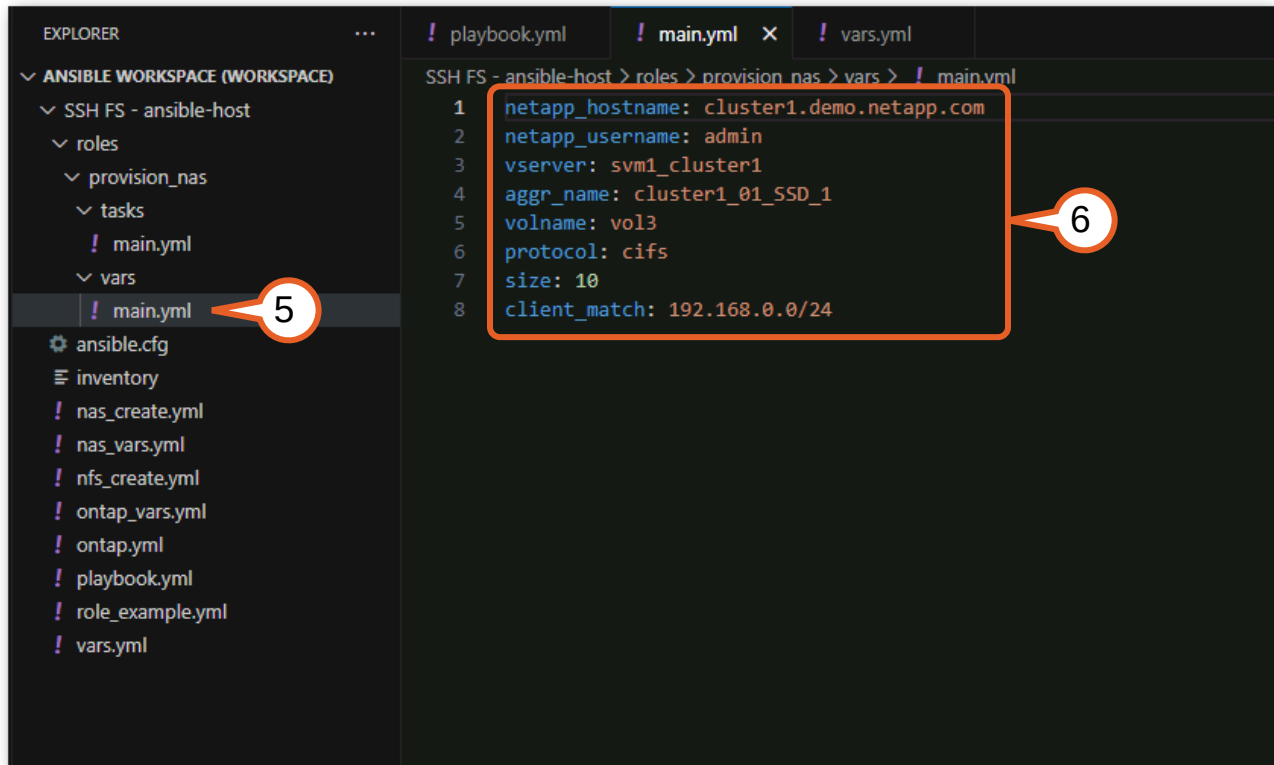


Figure 2-48:

7. Now click **role_example.yml** from the left panel.
8. The **role_example.yml** playbook opens in a tab on the right.

This playbook includes a `vars_prompt` for the netapp password, but otherwise does not include any of the unique elements from our provisioning playbook. Instead, the playbook references the `provision_nas` role and uses it twice.

Each use of the role is accompanied by variable definitions for `volname` and `protocol`. Even though those variables were included in the `vars` file in the role, defining them here takes precedence and overrides the values in the file.

By using the role in this way, in a single playbook you can easily provision two volumes with different qualities, one NFS, and one CIFS.



Tip: This playbook could include additional roles, as well as tasks that are executed after the roles. Remember, a single Ansible playbook can make changes to a variety of components in your IT environment, from networking to cloud resources.

Using roles makes it easier to work between different contexts by allowing you to isolate the automation of each component as its own role.

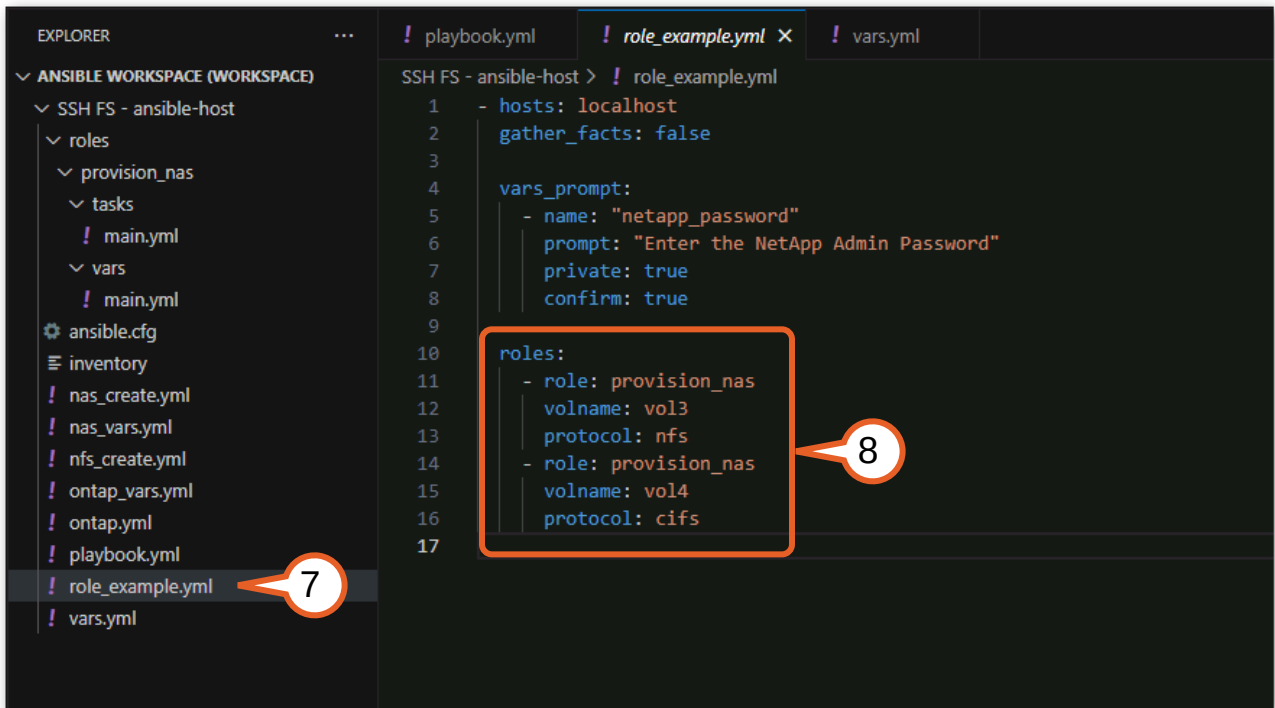


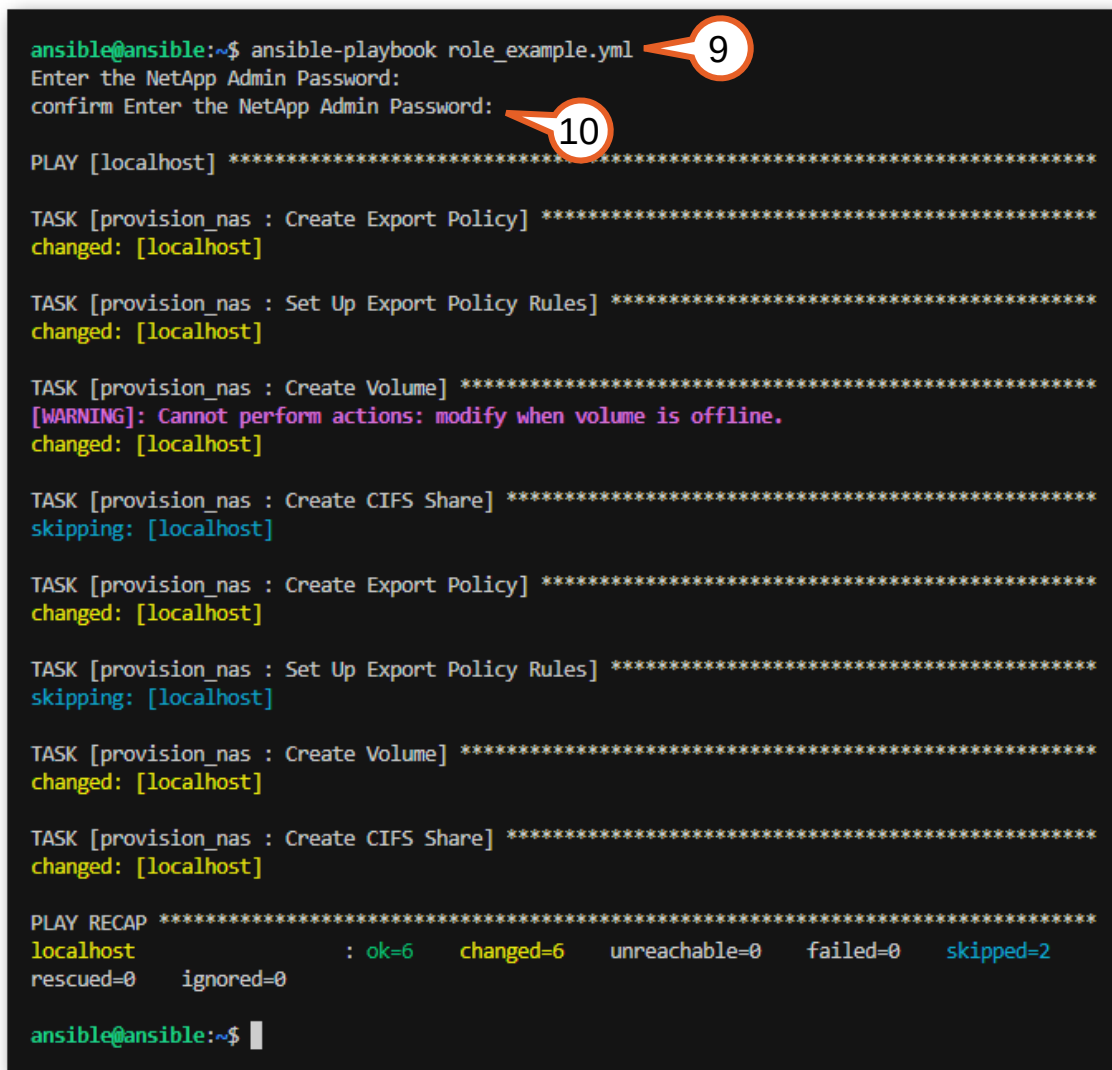
Figure 2-49:

9. In the terminal window below, run the playbook:

```
ansible-playbook role_example.yml
```

10. When prompted for the password, enter **Netapp1!** and press **enter**. You will need to do this twice.

11. The result is the equivalent of running your NAS provisioning playbook twice, once with an NFS volume and once with a CIFS volume.



```
ansible@ansible:~$ ansible-playbook role_example.yml
Enter the NetApp Admin Password:
confirm Enter the NetApp Admin Password:

PLAY [localhost]

TASK [provision_nas : Create Export Policy]
changed: [localhost]

TASK [provision_nas : Set Up Export Policy Rules]
changed: [localhost]

TASK [provision_nas : Create Volume]
[WARNING]: Cannot perform actions: modify when volume is offline.
changed: [localhost]

TASK [provision_nas : Create CIFS Share]
skipping: [localhost]

TASK [provision_nas : Create Export Policy]
changed: [localhost]

TASK [provision_nas : Set Up Export Policy Rules]
skipping: [localhost]

TASK [provision_nas : Create Volume]
changed: [localhost]

TASK [provision_nas : Create CIFS Share]
changed: [localhost]

PLAY RECAP
localhost : ok=6  changed=6  unreachable=0  failed=0  skipped=2
rescued=0  ignored=0

ansible@ansible:~$
```

Callout 9 points to the command `ansible-playbook role_example.yml`. Callout 10 points to the password prompt `confirm Enter the NetApp Admin Password:`.

Figure 2-50:

Takeaway: Roles make it easy to build a library of automation scripts that you can chain together into more complex workflows.

3 Lab Summary

In this lab, you have seen how automating ONTAP administration with playbooks using Ansible-certified NetApp modules saves you time, prevents mistakes, and helps you tackle large tasks efficiently. Ansible features powerful automation capabilities, and with the `netapp.ontap` module collection, you can take full advantage of these features with your ONTAP storage.

4 Additional Information

4.1 Lab Environment

All of the servers and storage controllers presented in this lab are virtual devices, and the networks that interconnect them are exclusive to your lab session. While we encourage you to follow the demonstration steps outlined in this lab guide, you are free to deviate from this guide and experiment with other ONTAP features that interest you. While the virtual storage controllers (vsims) used in this lab offer nearly all of the same functionality as physical storage controllers, they are not capable of providing the same performance as a physical controller, so they are not suitable for performance testing.

The Lab Host Credentials table provides a list of the servers and storage controller nodes in the lab, along with their IP addresses.

Table 3: Lab Host Credentials

Hostname	Description	IP Address(es)	Username	Password
Jumphost	Windows Server 2019 Remote Access host	192.168.0.5	DEMO\Administrator	Netapp1!
Cluster1	NetApp ONTAP 9.16.1 cluster	192.168.0.101	admin	Netapp1!
Cluster2	NetApp ONTAP 9.16.1 cluster	192.168.0.102	admin	Netapp1!
DC1	Active Directory Server	192.168.0.253	DEMO\Administrator	Netapp1!
AWX	Ansible AWX host	192.168.0.241	ansible	Netapp1!
Ansible	Ansible host	192.168.0.188	ansible	Netapp1!
Ubuntu	Ubuntu Linux 22	192.168.0.61	ansible	Netapp1!
Win1	Windows Server 2019	192.168.0.40	DEMO\Administrator	Netapp1!

4.2 References

The following references were used in writing this lab guide.

- [NetApp Modules for Ansible Documentation](#)
- [NetApp.io: Configuration Management & Automation](#)
- **General Ansible References**
 - [Installation](#)
 - [How to Build Your Inventory](#)
 - [Ansible Playbooks](#)
 - [Variables](#)
 - [Conditionals](#)
 - [Loops](#)
 - [Anchors and Aliases](#)
 - [Ansible Vault](#)
 - [Ansible Roles](#)
 - [Ansible AWX Project on GitHub](#)

4.3 Version History

Version	Date	Document Version History
1.0	October 2023	Initial Release
1.1	October 2024	Updated to 9.15.1
1.2	September 2025	Updated to 9.16.1, removed AWX guide

Legal Notices

 **Note:** This document was created using machine translation. If you have any issues with this translated content please consult the English version.

Legal notices provide access to copyright statements, trademarks, patents, and more.

- **Copyright**

<http://www.netapp.com/us/legal/copyright.aspx>

- **Trademarks**

NETAPP, the NETAPP logo, and the marks listed on the NetApp Trademarks page are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.

<http://www.netapp.com/us/legal/netapptmlist.aspx>

- **Patents**

A current list of NetApp owned patents can be found at

<https://www.netapp.com/us/media/patents-page.pdf>

- **Privacy Policy**

<https://www.netapp.com/us/legal/privacypolicy/index.aspx>

- **Open Source**

Provides information about third-party copyright and licenses used in this product.

- [Notice for ONTAP 9.14.1](#)
- [Notice for ONTAP 9.14.0](#)
- [Notice for ONTAP 9.13.1](#)
- [Notice for ONTAP 9.12.1](#)
- [Notice for ONTAP 9.12.0](#)
- [Notice for ONTAP 9.11.1](#)
- [Notice for ONTAP 9.10.1](#)
- [Notice for ONTAP 9.9.1](#)
- [Notice for ONTAP 9.8](#)
- [Notice for ONTAP 9.7](#)

